

The Thiele Machine

A Mathematical Reference

Derived from the Machine-Checked Coq Corpus

Devon Thiele

Before the specification. I don't do math by hand. I think in shapes, directions, and connections, and my part of this work was to see where things had to fit, to direct, and to refuse what didn't fit. This document is the translation. Language models wrote the formal statements and the proofs, Coq checked every one of them, and I read the statements to make sure they say what the shape says. So nothing below is in my voice, on purpose. It's the reference layer: exact statements, exact premises, and where each one lives. If you want the pictures, the monograph starts with them. If you want to know exactly what is claimed, it's here.

The Thiele Machine is an abstract model of computation, and an argument made with it about what an account of computation should preserve.

It is a machine in the sense a Turing machine is one: a mathematical object, not a device. The 51-opcode VM summarized in the chapters below is one build of that model; its complete checked definition is the Coq source. It is not the model. The model is the abstract `CertificationSystem` and `Substrate` interfaces; the VM is one instance of each. The VM prices commitments and does not check that they were earned. A second, minimal machine, `minimal/EarnedCore.v`, is a two-counter machine whose commitments are earned (Remark 4.3).

Certification is one concrete witness of a broader investigation of structural state, observation, and cost. The abstract laws, theorems, and executable realizations belong to that investigation. Selected projections lose distinctions that the richer state can express; cost laws constrain specified events under explicit hypotheses.

A conventional encoding can preserve those distinctions. A projection that discards them cannot recover them in general. The stronger proposal that these distinctions belong in a necessary foundation of computation is a conjecture. Neither a physical cost metric nor infinitely many independent geometric dimensions follows from the existence of many observables. The status of each result is stated below.

Abstract

This is a mathematical reference for the Thiele Machine: a formal model of computation where structural information carries an explicit, monotone cost μ that starts at zero and never decreases. It presents the abstract interfaces, the main theorems, and selected semantics from the machine-checked Coq corpus. The complete 51-opcode operational definition is the Coq source; this document alone does not enumerate enough semantics to reconstruct it.

Coq establishes that conclusions follow from axioms. It does *not* establish that axioms model physical reality. That distinction is kept explicit throughout. Every result is tagged as one of three types: **(S)** structural theorems about the machine as a mathematical object, **(C)** conditional derivations where a physics conclusion follows from a stated axiom, and **(R)** consistency relations that check internal coherence but are not independent predictions.

The core claims are structural (S): No Free Insight; μ -conservation; μ -hierarchy; μ -ledger necessity (five-way independence classification); categorical separation; strict Turing subsumption of instruction sets; the classical CHSH bound ($|S| \leq 2$); the algebraic Tsirelson bound ($|S| \leq 2\sqrt{2}$); and, on the zero-marginal slice, the equivalence of column contractivity with positive semidefiniteness of one fixed 5×5 NPA matrix, with no project-local axioms `column_contractive_iff_npa_psd`. The CHSH algebraic result is pure math. The Landauer connection is motivation. Geometry: the discrete angle-defect identity under its incidence assumptions and the additional restriction $B = 3\chi$, as statements about partition graphs in general; on states the VM reaches, module regions are disjoint and these statements hold only in a degenerate form (Remark 19.3).

The document also covers, with explicit epistemological tags, conditional derivations under named physical axioms (Born rule under linearity, no-cloning, unitarity) and reference explorations (4D simplicial-complex differential geometry, self-reference tower). These are marked (C) or (R) and do not form part of the primary claim ledger. Two named trust boundaries discussed here are: `bsc_kami_compilation_trusted` (Layer 2 Verilog semantics gap) and `A_QM` (physical quantum axiom).

Contents

1	Introduction	4
1.1	Epistemological Framework	5
2	Notation and Conventions	6
2.1	Overloaded symbols	6
2.2	Terms used before definition	7
3	The State Space	8
3.1	Primitives	8
3.2	Regions and Modules	8
3.3	The Partition Graph	8
3.4	Machine State	8
4	Instruction Set and Cost Model	10
4.1	The Axiom Scheme A1–A6	10
4.1.1	The consolidated <code>CertificationSystem</code> (A1–A2)	10
4.1.2	The <code>QuantitativeCertificationSystem</code> extension (A3–A6)	10
4.2	Instruction Categories	11
4.2.1	Structural Operations	11
4.2.2	Logical Operations	12
4.2.3	Computational Operations	12
4.2.4	Standard ALU and Control Flow	12
4.2.5	Auxiliary Operations	13
4.2.6	Categorical Morphism Operations	13
4.3	Cost Model	14
4.4	The <code>CERTIFY</code> instruction	14
5	Operational Semantics	15
5.1	Transition Function	15
5.2	Traces and Execution	15
5.3	Selected Instruction Semantics	16
6	No Free Insight	17
6.1	Definitions	17
6.2	The Theorem	17
6.3	μ -Ledger Necessity	18
6.4	Partition Refinement and No Free Insight	19
7	The μ-Chaitin Theorem	21

8	Computational Universality and Subsumption	22
8.1	Strict Containment	22
8.2	Oracle Cost Lower Bound	22
8.3	μ -budget predicates	23
8.4	Intrinsic level hierarchy	24
8.5	Verifier model and the bare-setting impossibility	24
9	μ-Conservation and Receipt Integrity	26
9.1	The μ -Ledger Identity	26
9.2	Receipt Integrity	26
10	Gauge Shifts of the Ledger and a Noether-Style Invariance	27
10.1	The Partial $\mathbb{Z}$ -Action (Gauge Group)	27
10.2	Gauge Invariance	27
11	The Born Rule	29
12	Unitarity	30
13	Purification	31
14	No-Cloning	32
15	The Tsirelson Bound	33
16	Witness Insight Taxonomy	34
17	Bell's Theorem in Morphism Coupling Language	35
18	Slice Coherence and the Zero-Marginal NPA Matrix: The Biconditional	36
18.1	The CHSH statistic S and its witness counts	36
18.2	The Elliptope Completion and Its Gate	37
18.3	The Pointer-Observable Criterion	38
18.3.1	The adversarial search	39
19	Spacetime Analogies: a μ-Pseudometric, Reachability Cones, and Discrete Geometry	41
19.1	The μ -Metric	41
19.2	Causal Cones	41
19.3	No-Signaling (Data Isolation)	41
19.4	Discrete Geometry: A Restricted Angle-Defect Identity	42
19.4.1	Cross-link in MuGravity Vocabulary	42
19.5	Four-Dimensional Discrete Riemannian Geometry (Reference Only, Not a Main Claim)	43
19.5.1	4D Simplicial Complex	43
19.5.2	Metric Tensor from μ -Costs	44
19.5.3	Discrete Differential Geometry	44
19.5.4	Curvature Tensors	45
19.5.5	Stress-Energy Tensor	45
19.5.6	Einstein's Field Equations	46
19.5.7	Affine Metric-Scaled Operator: Non-Uniform Boundary Results	48
20	Information Causality	50

21 Thermodynamic Bridge: Conditional Links and Their Limits	51
21.1 Landauer’s Principle	51
21.2 Two-State Calorimeter Protocol	51
21.3 A2 from Landauer (Factored Form): a Conditional Whose Premises Fail on the Full ISA	52
21.4 Permanent Certification Forces a Merge	53
21.5 Exact Pricing Cannot Be Computed From a Window	55
21.6 Which Narrowing a Merge Price Charges	55
21.7 A Recursion Theorem for L	56
21.8 No Recursion Theorem for the Full VM	56
21.9 The Dimensional Gap: Why Ledgers Cannot Fix Constants	56
22 Self-Reference and the Thiele Manifold	58
22.1 Self-Reference Escalation	58
22.2 The Thiele Manifold	58
23 Three-Layer Observable Agreement	60
23.1 RTL Gap Registry and Inventory	60
23.2 The CPU Against the Kernel	61
23.3 Named Trust Boundaries	61
24 Categorical Structure	63
24.1 The Morphism Layer	63
24.1.1 Objects and Morphisms	63
24.1.2 The Seven Categorical Opcodes	63
24.1.3 Category Laws	64
24.1.4 The Categorical Separation Theorem	64
25 Falsifiable Predictions	66
25.1 Falsify No Free Insight	66
25.2 Falsify the Classical CHSH Bound	66
25.3 Falsify the Biconditional	66
25.4 Falsify the μ -Hierarchy	66
25.5 Boundaries and conjectures	66
A Proof File Index	68
B Epistemological Classification	71

Chapter 1

Introduction

The Thiele Machine makes the cost of structure explicit. Ordinary step-counting assigns one time unit per transition; it does not specify this structural-cost schedule. The Thiele Machine assigns a specific cost μ to every operation that extracts or asserts structural information. The chapters below lay out the consequences.

The central claim, proven formally: *specified certification and structural-entitlement events have the cost floors proved under their stated hypotheses*. Alongside that core, the specification records conditional results in which explicitly supplied premises (linearity, an information-conservation inequality, a second-law record field) yield statements with the shape of results in quantum mechanics and thermodynamics. In most of them μ enters as a named cost field, and the supplied premises do the work. They are marked **(C)**: conditional on those premises, not derived from the ledger alone. Each result’s status, whether a structural theorem **(S)**, a conditional derivation **(C)**, or a consistency relation **(R)**, is marked directly.

The document is organized as:

1. **Foundations:** State space, instruction set, operational semantics.
2. **Core Theorems:** No Free Insight, μ -ledger necessity, μ -Chaitin, μ -conservation.
3. **Computation:** Strict Turing subsumption of instruction sets, oracle pricing as instantiation of universal NFI, μ -complexity classes, μ -cost hierarchy.
4. **Symmetry & Conservation:** Gauge shifts of μ , a Noether-style invariance, receipts.
5. **Quantum / CHSH algebra:** Tsirelson bound (algebraic), CHSH classical bound, biconditional column-contractive $\iff$ PSD of the zero-marginal NPA matrix, three-tier observation taxonomy. Pure polynomial arithmetic, zero physics axioms.
6. **Conditional physics:** Born rule (under linearity), unitarity, no-cloning, purification, Landauer (with second-law assumed as a record field), affine EFE family. Each is conditional on stated axioms in its own file.
7. **Discrete Geometry:** Restricted angle-defect identity with $B = 3\chi$ (not classical Gauss-Bonnet); 4D simplicial complex and Einstein-tensor definitions (reference exploration only, vacuum case is a $0 = 0$ consistency check).
8. **Meta-Theory:** Three-layer Coq/Python/RTL observable contract, categorical structure.
9. **Predictions:** Falsifiable predictions tied to specific Coq-verifiable claims.

1.1 Epistemological Framework

Formal verification (Coq) establishes that conclusions follow logically from axioms. It does **not** establish that axioms model physical reality. This distinction is critical for evaluating the physics claims in Parts V–VIII.

Every result in this specification falls into one of three categories:

- (S) **Structural** Theorems about the model and the kernel as mathematical objects. Their conclusions follow from the definitions, explicit hypotheses, and reported library assumptions. *Examples:* μ -conservation, gauge invariance, strict Turing subsumption of instruction sets, and, for the calculus L, confluence of value reduction (`star_value_confluent`) and halting undecidability (`L_halting_undecidable`).
- (C) **Conditional** Theorems of the form “if axiom X holds, then physics result Y follows.” The logical inference is verified; whether axiom X holds in nature is an empirical question. *Examples:* Born rule uniqueness (conditional on linearity), complex necessity (conditional on 2D amplitudes), unitarity (conditional on information conservation).
- (R) **Consistency** Algebraic identities or definitional equivalences that verify internal coherence but do not constitute independent predictions. *Example:* the Tsirelson cost definition.

Each major theorem in Parts V–VIII is annotated with its category. A theorem marked (C) or (R) is not diminished. Conditional derivations and consistency checks are valuable scientific tools. But the reader should not mistake them for derivations from first principles without stated assumptions. See Appendix B for the complete classification table.

Chapter 2

Notation and Conventions

This chapter exists because several symbols in this document are overloaded, and at least one (S) is used well before it is defined. Read it once; it resolves ambiguities that would otherwise require guessing from context.

2.1 Overloaded symbols

S , **four distinct meanings**. Disambiguated by context and, where ambiguity would be real, by explicit qualification:

1. $\mathcal{S}$ (calligraphic): the *state space*.
2. S (roman, in operational-semantics chapters): a *particular VM state*, an element of $\mathcal{S}$.
3. $S(n)$ (applied to a natural number): the *successor function* on $\mathbb{N}$, i.e. $S(n) = n+1$. This is Coq's $\mathbf{S}$ constructor, and it is the reason cost expressions such as $\text{flen} \times 8 + S(\text{cost})$ are bounded below by 1: $S(n) \geq 1$ for every $n \in \mathbb{N}$. **This is the notation used from Chapter 4 onward, before its first formal appearance in the cost tables.**
4. S (in quantum chapters): the *CHSH statistic*, defined in Section 18.1. Real-valued.

G , **three distinct meanings**. All three occur, and two occur inside the same chapter on emergent geometry:

1. G : the *partition graph* component of a VM state.
2. G : *Newton's constant*, fixed to $1/(8\pi)$ as a unit choice.
3. $G_{\mu\nu}$: the *Einstein tensor*.

Where the geometry chapter needs the partition graph it is written G_{part} .

μ , **ledger and index, and two different number systems**.
1. μ : the *cost ledger*, a component of VM state. In the machine, μ **ranges over** $\mathbb{N}$: it is a natural number, it is monotone non-decreasing under the step relation, and A2 gives it an integer floor.
2. μ, ν as *tensor indices* in the geometry chapter ($g_{\mu\nu}, T_{\mu\nu}$). Unrelated to the ledger.
3. μ in the **quantum chapters** (measurement cost, the no-cloning inequality, the μ/I ratio) is **real-valued**. This is a genuinely different object from the machine's μ : there is no theorem in this development identifying them, and no coercion between them is claimed. Where the distinction matters the real-valued quantity is written $\mu_{\mathbb{R}}$.

2.2 Terms used before definition

CERTIFY. The instruction family whose step sets the certification bit. Formally $\text{CERTIFY}(\Delta\mu)$ has step semantics $\text{cert} \mapsto \text{true}$ with cost $S(\Delta\mu) \geq 1$; see Section 4.4. Chapters 5, 7 and 8 depend on it.

irreversible_bits. Zero when the instruction's cost is zero, one otherwise (`coq/kernel/foundation/MuLedgerConservation.v`). It is a per-instruction counter tied to the cost model; it does not measure whether a prior value is recoverable. Used in the thermodynamic corollaries.

has_structure_addition. At least one executed step within the supplied fuel changes the certification CSR from zero to non-zero, as defined in `coq/kernel/nfi/NoFreeInsight.v`.

Chapter 3

The State Space

3.1 Primitives

Let $\mathbb{N} = \{0, 1, 2, \dots\}$. Let $\mathbb{B} = \{0, 1\}$. Let Σ be a finite alphabet.

3.2 Regions and Modules

Definition 3.1 (Region). A **Region** R is a finite subset of $\mathbb{N}$, canonically represented as a deduplicated list preserving first-occurrence order: $\text{norm}(L) = \text{nodup}(L)$ removes duplicate elements while preserving the order of their first appearance. (The Python VM applies a stricter canonical form via `sorted(set(region))`; the Coq specification uses `nodup Nat.eq_dec` which does not sort.)

Definition 3.2 (Module). A **Module** $M = (R_M, A_M, T_M)$ consists of a normalized region $R_M \subset \mathbb{N}$, a set of axioms $A_M \subset \Sigma^*$, and a per-module 4×4 μ -tensor T_M (16 natural-number entries, row-major; `module_mu_tensor`).

3.3 The Partition Graph

Definition 3.3 (Partition Graph). A partition graph $G = (N_{id}, \mathcal{T}, N_{mor}, \mathcal{F})$ where $N_{id} \in \mathbb{N}$ is the next available module ID, $\mathcal{T} : \{0, \dots, N_{id} - 1\} \rightarrow \mathcal{M}$ is a finite partial function from IDs to modules, $N_{mor} \in \mathbb{N}$ is the next available morphism ID, and $\mathcal{F}$ is a finite partial function from morphism IDs below N_{mor} to morphisms (a source module, a target module, and coupling data).

Definition 3.4 (Well-Formedness). G is well-formed iff $\text{dom}(\mathcal{T}) \subseteq \{0, \dots, N_{id} - 1\}$, $\text{dom}(\mathcal{F}) \subseteq \{0, \dots, N_{mor} - 1\}$, and the source and target of every morphism in $\mathcal{F}$ are in $\text{dom}(\mathcal{T})$ (`well_formed_graph`).

3.4 Machine State

Definition 3.5 (VM State). The complete state is a 12-tuple:

$$S = (G, C, R, Mem, PC, \mu, T_\mu, err, \ell, m, w, cert)$$

where $G \in \mathcal{G}$ is the partition graph, $C \in \mathbb{N}^4$ comprises the CSRs (certificate address, status, error code, heap base), $R \in \mathbb{N}^{\text{REG_COUNT}}$ is the register file, $Mem \in \mathbb{N}^{\text{MEM_SIZE}}$ is main memory (the kernel fixes `REG\_COUNT` = 16 and `MEM\_SIZE` = 128, and some kernel proofs are stated parametrically in them; register and memory words are truncated to 64 bits), $PC \in \mathbb{N}$ is the program counter, $\mu \in \mathbb{N}$ is the μ -ledger, $T_\mu \in \mathbb{N}^{16}$ is the flattened 4×4 μ -tensor (row-major),

$err \in \mathbb{B}$ is the global error flag (latching), $\ell \in \mathbb{N}$ is the logic-engine accumulator, $m \in \mathbb{N}$ is the machine status register (0 = Turing mode, 1 = Thiele mode), $w \in \mathbb{N}^8$ is the WitnessCount record (**vm_witness**): eight natural-number counters **wc_same_ab** and **wc_diff_ab** for each setting pair $(a, b) \in \{00, 01, 10, 11\}$, and $cert \in \mathbb{B}$ is the certified flag (**vm_certified**).

Chapter 4

Instruction Set and Cost Model

4.1 The Axiom Scheme A1–A6

“A2” is referred to throughout this document, and this is the scheme it belongs to. These are *conditions on Coq record types*, not **Axiom** declarations. The assumption receipt distinguishes project-local declarations from standard-library axioms and explicit theorem hypotheses. Supplying an instance of the record is the only way to invoke the corresponding theorems, and the 51-opcode VM supplies them by construction.

4.1.1 The consolidated **CertificationSystem** (A1–A2)

A **CertificationSystem** is a tuple $(\Sigma, \mathcal{I}, \text{step}, \text{cost}, \text{cert})$ where Σ is any state type, $\mathcal{I}$ any instruction type, $\text{step} : \Sigma \times \mathcal{I} \rightarrow \Sigma$, $\text{cost} : \mathcal{I} \rightarrow \mathbb{N}$, and $\text{cert} : \Sigma \rightarrow \mathbb{B}$, subject to:

A1 (certification is observable). A total function $\text{cert} : \Sigma \rightarrow \mathbb{B}$ exists. This holds *by record existence*: providing the field is providing the axiom.

A2 (certification is priced; the load-bearing one).

$$\forall s \in \Sigma, \forall i \in \mathcal{I}, \quad \text{cert}(s) = \text{false} \wedge \text{cert}(\text{step}(s, i)) = \text{true} \implies \text{cost}(i) \geq 1. \quad (4.1)$$

In Coq this is the field `cs_cert_costs`. Everything called “No Free Insight” in this document is A2 lifted from steps to traces by induction; the trace-level statement is genuinely proved, but its content is A2 plus “a sum of naturals containing a summand ≥ 1 is ≥ 1 ”.

Remark 4.1 (A2 is a stipulation, and that is the intended reading). A2 is a condition a substrate may or may not satisfy, not a discovered fact about computation. Nothing in this document derives it; Section 21.3 derives it *conditionally* from a Landauer premise plus a named calibration bridge, which relocates the stipulation rather than removing it. Claims of the form “the machine proves certification must cost” should be read as “on any substrate satisfying A2, certification costs”. That is what `universal_nfi_any_substrate` states, over an arbitrary state type and an arbitrary instruction type.

4.1.2 The **QuantitativeCertificationSystem** extension (A3–A6)

The quantitative extension lifts the floor from ≥ 1 to $\geq K$ for an arbitrary threshold. It adds a witness function $W : \Sigma \rightarrow \mathbb{N}$ and a threshold $K \in \mathbb{N}$, subject to:

A3 (cost bounds witness growth). $\forall s, i : W(s) + \text{cost}(i) \geq W(\text{step}(s, i))$. Evidence cannot accumulate faster than it is paid for. Coq: `qcs_cost_bounds_witness`.

A4 (the witness is nondecreasing). $\forall s, i : W(s) \leq W(\text{step}(s, i))$. It does not follow from A3, which bounds growth from above. It is a separate field of the extended record (`qcsf_witness_nondecreasing`) and the floor does not use it.

A5 (certification requires the threshold). $\forall s : \text{cert}(s) = \text{true} \implies W(s) \geq K$. Coq: `qcs_cert_threshold_witness`.

A6 (clean start). The initial state carries no witness, $W(s_0) = 0$. Supplied as a hypothesis at use sites rather than as a record field. With A3 and A5 it gives the conclusion: a run from s_0 to a certified state has total cost $\geq K$.

Remark 4.2 (Two numbering systems, and which one this document uses). The record `AbstractCertMachine` has its own labels A1–A4, in which **A4** is the cost-on-certification rule and **A2** is “cert starts unset”. The labels A3–A6 above follow `QuantitativeNoFI.v`. **Throughout this document, “A2” means the cost-on-certification rule in the consolidated CertificationSystem sense stated above.** When reading kernel sources that mention `AbstractCertMachine`, translate accordingly.

4.2 Instruction Categories

The instruction set $\mathcal{I}$ is organized into six families. Every instruction i carries an explicit cost parameter $\Delta\mu_i \in \mathbb{N}$.

4.2.1 Structural Operations

These modify the partition graph G :

- **PNEW**($R, \Delta\mu$): Claim the range of data memory that starts at the first address of `norm(R)` and has $|\text{norm}(R)|$ addresses (`pnew_region`). If a module already owns exactly that range, the graph is unchanged; otherwise a fresh module with no axioms takes it.
- **PSPLIT**($m, L, R, \Delta\mu$): Remove module $m \bmod 64$ and the arrows that touch it, and add two fresh modules: the first $\lfloor n/2 \rfloor$ addresses of its range and the rest. L and R are accepted and ignored.
- **PMERGE**($m_1, m_2, \Delta\mu$): Remove modules $m_1 \bmod 64$ and $m_2 \bmod 64$ and the arrows that touch them, and add one fresh module whose range is the two ranges joined.
- **PDISCOVER**($m, E, \Delta\mu$): Discovery accounting for module m . The abstract specification attaches evidence E (list of axioms) to the module; the hardware-aligned step advances PC and charges $\Delta\mu$ without mutating the graph.
- **MDLACC**($m, \Delta\mu$): Advance the PC and charge $\Delta\mu$; nothing else changes.

PNEW succeeds only when a module number is free ($N_{id} + 1 \leq 64$), every claimed address is below `MEM_SIZE` = 128, and the range overlaps no module except one whose range it is (`pnew_ok`). **PSPLIT** needs two free module numbers (`module_room`). **PMERGE** needs one free number and two ranges that, laid end to end in one order or the other, form one range (`pmerge_ok`). When the check fails the step traps: $err \leftarrow \text{true}$, the error code is set, $PC \leftarrow \text{LASSERT_TRAP_PC}$, the cost is charged, and the graph is unchanged. From a well-formed graph with no modules and a module counter at most 64, every reachable graph has pairwise disjoint, contiguous regions inside data memory, at most 64 modules, and every module number below 64, so reading a module number modulo 64 changes no live number (`vm_reachable_partition_in_bounds`).

4.2.2 Logical Operations

These interact with information content:

- **LASSERT**($freg, creg, kind, flen, cost$): Assert a formula read from `vm_mem[R[freg]]` (encoded-unit count $flen$, eight priced bits per unit; the factor 8 is a pricing convention) against a witness package at `vm_mem[R[creg]]`. In SAT mode the package contains one satisfying assignment and one falsifying assignment, stored back-to-back; $\Delta\mu = flen \times 8 + S(cost) \geq 1$. The step succeeds only when $flen$ matches the header length of the formula in memory (`lassert_hw_flen`); a mismatch traps like a failed check.
- **LJOIN**($c1reg, c2reg, \Delta\mu$): Advance the PC and charge $S(\Delta\mu)$. It neither reads nor compares the certificate strings.
- **REVEAL**($m, n, cert, \Delta\mu$): Add n to the μ -tensor entry at index $m \bmod 16$, advance the PC, and charge $n + S(\Delta\mu)$. The certificate string is not examined.
- **EMIT**($m, p, \Delta\mu$): Emit payload p from module m ; cost is $|p|_{\text{bits}} + S(\Delta\mu)$.

4.2.3 Computational Operations

Register/memory operations:

- **XFER**($dst, src, \Delta\mu$): Copy register src to dst .
- **XOR_LOAD**($dst, addr, \Delta\mu$): Load memory word at $addr$ (a plain load; no XOR is involved).
- **XOR_ADD**($dst, src, \Delta\mu$): $dst \leftarrow dst \oplus src$.
- **XOR_SWAP**($a, b, \Delta\mu$): Swap registers a and b .
- **XOR_RANK**($dst, src, \Delta\mu$): $dst \leftarrow \text{popcount}(src)$.

4.2.4 Standard ALU and Control Flow

- **LOAD_IMM**($dst, imm, \Delta\mu$): Load the immediate, truncated to 64 bits, into register.
- **LOAD**($dst, addr, \Delta\mu$): Load from main memory.
- **STORE**($addr, src, \Delta\mu$): Store to main memory.
- **ADD**($dst, r_1, r_2, \Delta\mu$): $dst \leftarrow r_1 + r_2 \pmod{2^{64}}$.
- **SUB**($dst, r_1, r_2, \Delta\mu$): $dst \leftarrow r_1 - r_2 \pmod{2^{64}}$.
- **JUMP**($target, \Delta\mu$): Unconditional jump.
- **JNEZ**($r, target, \Delta\mu$): Jump if $R[r] \neq 0$.
- **CALL**($target, \Delta\mu$): Push return address, jump to target.
- **RET**($\Delta\mu$): Pop return address, jump back.

4.2.5 Auxiliary Operations

- `CHSH_TRIAL`($x, y, a, b, \Delta\mu$): Run a CHSH correlation trial; $x, y, a, b \in \{0, 1\}$ required.
- `CHECKPOINT`($label, \Delta\mu$): Execution checkpoint (charges μ).
- `READ_PORT`($dst, ch, val, bits, \Delta\mu$): Read from I/O channel (positive-cost class).
- `WRITE_PORT`($ch, src, \Delta\mu$): Write to I/O channel.
- `HEAP_LOAD`($dst, addr, \Delta\mu$): Load from heap-relative address.
- `HEAP_STORE`($addr, src, \Delta\mu$): Store to heap-relative address.
- `HALT`($\Delta\mu$): Advance the PC and charge $\Delta\mu$; execution ends only when the PC leaves the program or the fuel runs out.

4.2.6 Categorical Morphism Operations

These 7 opcodes (0x27–0x2D) implement the categorical layer over partition modules. See Chapter 24 for laws and proofs.

- `MORPH`($dst, src, tgt, idx, \Delta\mu$): Create morphism $src \rightarrow tgt$; write morphism ID to `regs[dst]`.
- `COMPOSE`($dst, f, g, \Delta\mu$): Compose $f; g$ when $f.tgt = g.src$; errors on type mismatch.
- `MORPH_ID`($dst, m, \Delta\mu$): Create identity morphism id_m (diagonal relation).
- `MORPH_DELETE`($f, \Delta\mu$): Remove morphism f ; cascades when modules are split/merged.
- `MORPH_ASSERT`($f, prop, cert, \Delta\mu$): **Cert-setter**. Charges $S(\Delta\mu) \geq 1$ unconditionally. Even failed attempts (morphism does not exist) incur cost 1. This is the strongest form of the NoFI law: the machine refuses to let even a doomed attempt at certifying a relational claim slip through for free.
- `MORPH_TENSOR`($dst, f, g, \Delta\mu$): Tensor product $f \otimes g$ (requires the union modules to exist, the source regions of f and g to be disjoint, and the target regions to be disjoint).
- `MORPH_GET`($dst, f, sel, \Delta\mu$): Read morphism metadata: selector 0 = src, 1 = tgt, 2 = coupling length, 3 = is_identity.

A reminder before the machinery: this ISA is scaffolding. It is one way to build the substrate, not the substrate itself. Burn all 51 opcodes and the machine is exactly as real, because the machine is the law, not the instruction set.

The active implementation exposes a 51-opcode ISA: 14 arithmetic/data, 8 memory and control flow, 7 I/O and tensor, 4 partition operations, 3 logic and certification, 8 witness and certification (`CHSH_TRIAL`, `CERTIFY`, `REVEAL`, `CHSH_LASSERT`, and the four Q_{1+AB} check variants), 7 categorical morphism operations ($14 + 8 + 7 + 4 + 3 + 8 + 7 = 51$). The CPU implements 47 of the 51. The four Q_{1+AB} variants are in `kami_step`, the Gallina model of the hardware step, whose step agrees with the kernel's, and the CPU has no opcode for them; they run in the kernel, the extracted runner and the Python VM. This condensed mathematical specification presents the core instruction families relevant to the kernel semantics and cost model. Every instruction carries an explicit $\Delta\mu$ parameter; the function `instruction_cost` extracts it.

4.3 Cost Model

Every instruction i carries an explicit cost parameter $\Delta\mu_i \in \mathbb{N}$. The function `instruction_cost( $i$ )` extracts this parameter. The cost is applied via `apply_cost( $s, i$ ) =  $s.\mu + \text{instruction\_cost}(i)$` . Since $\Delta\mu_i : \mathbb{N}$, the μ -ledger is monotonically non-decreasing by construction.

The mandatory positive-cost class is: `CERTIFY`, `MORPH_ASSERT`, and `LJOIN` (each charges $S(\Delta\mu) \geq 1$); `EMIT` ($|payload|_{\text{bits}} + S(\Delta\mu)$); `REVEAL` and `READ_PORT` ($bits + S(\Delta\mu)$); and `LASSERT` ($flen \times 8 + S(\Delta\mu)$). These cannot be zero-cost even with $\Delta\mu = 0$. The five `CHSH_LASSERT` family instructions also charge $S(\Delta\mu)$. Other instructions charge their encoded $\Delta\mu$, which may be zero.

Theorem 4.1 (Least joint unit floor, (S)). *For a finite family of event predicates on instructions, the least natural-valued schedule satisfying a unit floor on each event is the indicator that at least one event occurs. This is `joint_floor_is_least`. Independent Boolean coordinates can change together for one unit under these rules; additivity requires an additional condition.*

4.4 The `CERTIFY` instruction

`CERTIFY` is referred to throughout Chapters 6 onward and is defined here.

Definition 4.1 (`CERTIFY` (`instr_certify`, `VMStep.v`)). `CERTIFY`($\Delta\mu$) has the step semantics

$$pc \mapsto pc + 1, \quad \mu \mapsto \mu + S(\Delta\mu), \quad cert \mapsto \text{true}, \quad (4.2)$$

with every other state component unchanged. Its cost is $S(\Delta\mu) \geq 1$, which is the concrete instance of A2 (Section 4.1) for this ISA. It is the *only* rule in `VMStep.v` that sets `cert := true`.

Remark 4.3 (`CERTIFY` is unguarded; what that does and does not mean). The rule fires for every state and every $\Delta\mu$: it carries no precondition, takes no proposition as an argument, and checks nothing. Setting the certification bit is therefore always *available*, at a price of at least 1. Conversely, the instruction that does perform a decidable check on data, namely `LASSERT`, which traps on failure, leaves both the certification CSR and `cert` unchanged.

The description of the mechanism is therefore *metering*, not verification: the step rule guarantees that flipping the bit is priced, not that the bit is only set when some claim holds. Statements of the form “this machine makes stamping *verified* impossible” overstate it; what is impossible is stamping it *for free*.

The ISA does contain genuinely guarded instructions. `MORPH_ASSERT` requires the named morphism to exist, and `CHSH_LASSERT` runs `column_contractive_check_witness` on actual witness data and traps on failure, with a soundness bridge to PSD. A substrate with a guarded certification bit is formalised separately, on the Coq standard library alone, in `minimal/EarnedCore.v`: a two-counter machine with a table of checked facts and three priced instructions. `CHECK` writes a fact (property, counter, the counter’s current version) only when the property holds; `COMMIT` succeeds only on such a fact at the counter’s current version; `CERTIFY` succeeds only after a commitment. From a clean start every certified run contains a passing check, a passing commitment of the same claim with the counter untouched in between, and a passing certification (`earned_certification_provenance`); a fact whose counter has not been written since its check names a true claim (`checker_soundness`); every certified run costs at least 3 (`certified_run_min_cost`). Its prices are still flat, one per instruction, so A2 holds there by the price list and is not derived from the cost of establishing the claim; see Remark 4.1. A host on the same machine, in `minimal/UniversalThiele.v`, runs any of its programs as a guest, keeps a mirror equal to the guest’s certified flag (`record_agreement`), and raises the mirror only in a guest step that passed `CERTIFY`, charged in that step (`toll_enforced_by_host_step`); a machine from elsewhere enters through the two-counter encoding with its computation and toll kept but not its own prices.

Chapter 5

Operational Semantics

5.1 Transition Function

The transition $\delta : \mathcal{S} \times \mathcal{I} \rightarrow \mathcal{S}$ is defined by:

1. $PC' = PC + 1$ for non-control-flow instructions; JUMP, JNEZ, CALL, and RET set PC to the target address; HALT advances PC like any other non-control-flow instruction.
2. $\mu' = \mu + \Delta\mu_i$.
3. $err' = err \vee \text{fail}(S, i)$.

5.2 Traces and Execution

Definition 5.1 (Trace). A trace $\tau = [i_0, \dots, i_k] \in \mathcal{I}^*$ is a finite sequence of instructions.

Definition 5.2 (Execution). $\text{Run}([], S) = S$; $\text{Run}(i :: \tau', S) = \text{Run}(\tau', \delta(S, i))$. This is `run_instrs` in `coq/NecessityOfMuLedger.v`, a fold of `vm_apply` over the list. The bounded executor `run_vm` (`SimulationProof.v`) takes a fuel bound, fetches the instruction at the current PC, and may skip or repeat instructions. Theorems stated with a fuel argument use `run_vm` or `trace_run` (`RevelationRequirement.v`); the abstract-substrate theorems fold over the list directly.

Theorem 5.1 (Unique descended values, (S)). *Let $V(t)$ be evaluation of a VM instruction list from `init_state`, and let $T(t)$ be target evaluation from a fixed basepoint. Then*

$$(\forall t, u, V(t) = V(u) \Rightarrow T(t) = T(u))$$

is equivalent to each reachable VM state having exactly one target value represented by its traces. This is `trace_descent_unique_value_iff`.

Theorem 5.2 (Constructive reachable simulation, (S)). *Supply a function r selecting an instruction list for each VM state, satisfying $V(r(V(t))) = V(t)$ for every trace t . A map preserving the basepoint, steps on reachable states, and certification exists exactly when target evaluation respects equal VM outcomes and target certification agrees with VM certification on every trace. The map is $s \mapsto T(r(s))$ and is pointwise unique on reachable states. This is `reachable_simulation_exists_iff` together with `reachable_simulation_unique`. A selector exists: `reachable_trace_representative` returns, for each state, the first trace in an enumeration of all traces that evaluates to it, found by search with the standard-library axiom `sig_forall_dec (reachable_trace_representative_correct)`. It is a search, not a program that rebuilds a trace from the state. No extension of the step law to unreachable states is asserted.*

The certification quotient gives an explicit inhabited target. Conversely, the trace-history target satisfies A2 and certification agreement but fails descent: the empty trace and `JUMP 0 0` reach the same VM state while leaving distinct histories.

5.3 Selected Instruction Semantics

PNEW: Claim the range `pnew_region( $R$ )`. If a module already owns exactly that range, return it and leave the graph unchanged; otherwise add a module with the next module ID, no axioms, and that range. The step traps instead when no module number is free, the range runs past data memory, or it overlaps another module’s range (`pnew_ok`).

REVEAL: Add *bits* to the μ -tensor entry at index (module mod 16); the CSRs are unchanged and the certificate *cert* is not examined. Cost: *bits* + $S(\Delta\mu)$.

LASSERT: Read formula from `vm_mem[R[formula_addr_reg]]` (encoded-unit count `formula_len`, eight priced bits per unit, a pricing convention) and witness package from `vm_mem[R[cert_addr_reg]]`. `cert_kind = true` (SAT): verify using a satisfying model plus a falsifying countermodel (dual-witness). `cert_kind = false` (UNSAT): the kernel step always fails at this path, `lassert_check_ok` returns `false` unconditionally; UNSAT certificates are not checked, and LRAT checking is not invoked in the step semantics. Success also requires `formula_len` to match the header length of the formula in memory (`lassert_hwflen`). If the check succeeds, the step advances the PC with no trap and no error latch, and no certification state changes; if it fails, or the length does not match, the PC jumps to `LASSERT_TRAP_PC` and `err` $\leftarrow$ `true`. LASSERT does not mutate the partition graph’s axiom set; cert-channel activation happens through the subsequent `CERTIFY` or `MORPH_ASSERT`. Cost: `formula_len` $\times$ 8 + $S(\mu_delta)$.

Chapter 6

No Free Insight

The central theorem: structural insight is never free.

6.1 Definitions

Definition 6.1 (Receipt Predicate). $P : \mathcal{O} \rightarrow \mathbb{B}$ is a computable function on observation histories.

Definition 6.2 (Strength Ordering). $P_1 \leq P_2$ iff $\forall o. P_1(o) = 1 \implies P_2(o) = 1$. $P_1 < P_2$ iff $P_1 \leq P_2$ and $\exists o. P_2(o) = 1 \wedge P_1(o) = 0$.

Definition 6.3 (Certification). Trace τ **certifies** P iff execution succeeds ($err = 0$), the supra-certificate flag is set, and the output satisfies P .

Definition 6.4 (Structure Addition). $\text{HasStructureAddition}(\tau)$ is true iff some step of the bounded run of τ changes the certification CSR `csr_cert_addr` from zero to non-zero (`has_structure_addition`). Membership of an instruction in τ is not execution. The opcode list `cert_addr_setterb` (REVEAL, EMIT, LJOIN, LASSERT, MORPH_ASSERT) is a positive-cost policy class and a superset of what writes the CSR.

6.2 The Theorem

Theorem 6.1 (No Free Insight (`NoFreeInsight_Theorem.v`)). *Let $P_{\text{strong}} < P_{\text{weak}}$. If a trace τ starting from a clean state s_0 certifies P_{strong} , then:*

$$\text{Certified}(\tau, P_{\text{strong}}) \implies \text{HasStructureAddition}(\tau)$$

Proof Sketch. The statement is parametric: `NoFreeInsight_Theorem.v` states it for any system satisfying the `NO_FREE_INSIGHT_SYSTEM` module type interface, by applying the interface's no-free-insight field. The interface requires four contract obligations (not Coq Axiom declarations, they are proved separately for the kernel instantiation): (1) μ -monotonicity, (2) a certification specification linking `certifies` to execution, (3) a no-free-insight contract stating that strict strengthening requires structure addition, and (4) that the system has a notion of clean start. The theorem file itself contains zero Axiom, zero Admitted. The load-bearing proof is the kernel instantiation (`coq/nofi/Instance_Kernel.v`). There the strictness hypothesis $P_{\text{strong}} < P_{\text{weak}}$ is not used. The content is: from `csr_cert_addr = 0`, a run to a certified state contains a step that changes `csr_cert_addr` from zero to non-zero. It is evidence that certification requires structure addition, not that strengthening does. The predicate-level form that uses strictness is `strengthening_requires_structure_addition` (`NoFreeInsight.v`), with premises `strictly_stronger` $P_{\text{strong}} P_{\text{weak}}$, initial `csr_cert_addr = 0`, and a certified state after `run_vm` with a fuel bound. $\square$

Corollary 6.2 (Cost of Insight). *From an initial state with $\text{csr_cert_addr} = 0$, a run of $\text{thiele_cert_machine}$ that ends with $\text{csr_cert_addr} \neq 0$ raises μ by at least 1 ($\text{partition_refinement_nonfree}$, $\text{PartitionRefinementNoFI.v}$).*

6.3 μ -Ledger Necessity

Theorem 6.3 (Observation adequacy, (S)). *For observation $o : S \rightarrow V$ and query $q : S \rightarrow A$, the existence of a decoder $d : V \rightarrow A$ satisfying $d(o(s)) = q(s)$ implies*

$$o(s) = o(t) \implies q(s) = q(t).$$

Given a section $r : V \rightarrow S$ with $o(r(v)) = v$, this condition is also sufficient: $d = q \circ r$. These are $\text{decoding_requires_fiber_constancy}$ and $\text{selected_representatives_give_decoder}$.

Theorem 6.4 (μ -Ledger Necessity and Complete Independence Classification (NecessityOf-MuLedger.v, kernel/NecessityAbstract.v), (S)). *Five independence results are formally proven, zero Admitted:*

1. $\mu \perp (\text{mem}, \text{regs}, \text{pc})$: no strict-classical oracle recovers μ *turing_ram_mu_necessity*
2. $\text{certified} \perp (\text{mem}, \text{regs}, \text{pc})$: no strict-classical oracle recovers certification *turing_ram_cert_necessity*
3. $\text{certified} \perp (\text{mem}, \text{regs}, \text{pc}, \mu)$: knowing μ does not determine certification *cost_model_cert_necessity*
4. $\mu \perp (\text{mem}, \text{regs}, \text{pc}, \text{certified})$: knowing certification does not determine μ *cert_model_mu_necessity*
5. $\text{vm_graph} \perp (\text{mem}, \text{regs}, \text{pc}, \mu, \text{certified})$: the categorical layer is independent of the full μ -ledger shadow *graph_not_recoverable_from_P_full*

Three-component independence ($\text{thiele_state_three_component_independence}$): μ , vm_certified , and vm_graph are the three provably independent non-classical state components. Each of μ , vm_certified , and vm_graph is irrecoverable from the specific projections listed in items 1 to 5 above. Not proved: recovery of μ or vm_certified from a projection that also includes vm_graph .

Non-droppability, not representation minimality ($\text{P_full_complete_neither_mu_nor_cert_droppable}$): $(\text{mem}, \text{regs}, \text{pc}, \mu, \text{certified})$ is complete for returning μ and certification, and neither named coordinate can be dropped from this projection while retaining its corresponding answer. No ordering over arbitrary encodings is defined, so this is not a smallest-representation theorem. Graph structure is a third independent axis.

Universality ($\text{mu_ledger_necessity_universal}$): The separation holds from every VM-State, not only from crafted zero-state witnesses. From any reachable state, $\text{CERTIFY } 0$ and $\text{PNEW } [] \ 0$ produce identical strict classical shadows but different μ . Certification also differs when the source state is uncertified.

Trace closure ($\text{po1_trace_necessity}$): For any common prefix program of any length, the two witness extensions maintain equal strict shadows but strictly different μ . No accumulation of classical computation closes the gap.

Proof Sketch. Each separation uses explicit witnesses where equal projected inputs force equal oracle outputs, while the witness states require unequal outputs, a contradiction. Strict shadow equality: both $\text{CERTIFY } 0$ and $\text{PNEW } [] \ 0$ advance pc by 1 and preserve mem/regs . μ divergence: CERTIFY charges $S(0) = 1$; PNEW charges 0. Graph separation uses $\text{categorical_separation}$

from `PartitionSeparation.v`: two states with identical P_{full} but different `pg_morphisms`. Universality follows because `vm_apply_certify_strict_shadow` and `vm_apply_pnew_strict_shadow` hold for arbitrary source states. $\square$

Theorem 6.5 (Latent μ : Trace Cost for a Fixed Schedule (`NecessityOfMuLedger.v` §7), (S)). *The μ -cost accumulated by any computation is a property of the instruction sequence, not of any tracking system. Four results, all zero Admitted:*

1. *`shadow_mu_is_computation_intrinsic`: For any program `prog` and any starting state `s`:*

$$(\text{run_instrs } s \text{ prog}).\mu = s.\mu + \text{trace_total_cost}(\text{prog})$$

The increment is determined entirely by the instruction sequence, independent of `vm_graph`, `vm_regs`, `vm_mem`, `vm_certified`, and the starting μ .

2. *`shadow_mu_delta_universal`: Two machines running the same program from any two starting states accumulate the same additional μ . The cost is written in the program before execution begins.*
3. *`shadow_mu_unique_accounting`: Any accounting M that increments by the schedule's cost on every instruction (*instruction_consistent*) and satisfies $M(s) = 0$ at the starting state assigns exactly `trace_total_cost` to the result. Replace the Thiele Machine's μ -counter with any other such accounting and you get the same number.*
4. *`shadow_mu_inevitable`: Any program containing an instruction with positive declared cost accumulates strictly positive μ from any starting state. This conclusion requires a positive-cost instruction.*

Interpretation. *Given a trace and this cost schedule, its sum is determined even if the observer does not retain the ledger. This does not establish a physical bill for every classical computation. A full encoding may retain the ledger; the irrecoverability theorem concerns the specified projection.*

6.4 Partition Refinement and No Free Insight

Theorem 6.6 (Partition Ops Are Free (`kernel/PartitionRefinementNoFI.v`), (S)). *PNEW, PSPLIT, and PMERGE are not cert-setters and can carry $\Delta\mu = 0$:*

- *`partition_structural_ops_not_cert_setters`: All three are outside the certificate-address setter class, which excludes them by definition.*
- *`partition_structural_ops_can_be_free`: All three can execute with $\Delta\mu = 0$.*
- *`partition_structural_trace_cannot_certify`: A trace containing only PNEW, PSPLIT, and PMERGE, at any costs, leaves `csr_cert_addr` where it started.*

Building partition structure costs nothing by itself. Certifying structural claims about that partition is what costs.

Theorem 6.7 (Partition Free But Certification Non-Free (`kernel/PartitionRefinementNoFI.v`), (S)). *`partition_free_but_certification_nonfree`: PNEW, PSPLIT, and PMERGE can each carry $\Delta\mu = 0$; none is a cert-address setter; and for every trace and every s_0 with `csr_cert_addr`(s_0) = 0, if the run of `thiele_cert_machine` ends with `csr_cert_addr` $\neq 0$, then $\mu_{\text{final}} \geq \mu(s_0) + 1$. Partition structure can be built at zero μ -cost; moving the certification address off zero cannot be done for free. Zero Admitted.*

Theorem 6.8 (Partition Refinement Non-Free (kernel/PartitionRefinementNoFI.v), **(S)**).
partition_refinement_nonfree: For s_0 with $\text{csr_cert_addr}(s_0) = 0$, any trace whose run of $\text{thiele_cert_machine}$ ends with $\text{csr_cert_addr} \neq 0$ contains a cert-address-setter instruction of cost at least 1 and raises μ by at least 1. This is the NoFI law specialized to the certification address. Zero Admitted.

Chapter 7

The μ -Chaitin Theorem

A Chaitin-style bound denominated in μ -currency. “Proves k bits” is an abstract predicate supplied by an instance of the interface; the theorem is not a statement about provability in a fixed formal system.

Theorem 7.1 (μ -Chaitin (MuChaitinTheory_TraceLocal.v)). *For any instance T of the interface `MU_CHAITIN_TRACE_SYSTEM`, the number of bits k the theory can certify is bounded by:*

$$k \leq |T| + c$$

where $|T|$ is the description length of the theory and c is a fixed overhead constant (`proves_bits_bounded_by_description_trace_local`). The proof chain: $k \leq \text{payload} \leq \mu\text{-info} \leq \text{budget} \leq |T| + c$. The description-length budget is an obligation of the interface, not derived. Pricing is required only of the instructions on the theory’s own traces (`priced_on_traces`), and the μ payment is derived from the kernel’s certification theorem.

The interface `MuChaitinTheory_Interface` (with `MuChaitinTheory_Theorem.v`) requires every instruction to be priced. The VM’s cost schedule does not meet that (`current_schedule_not_globally_cert_priced`), so that interface has no VM instance. The trace-local interface has one, `KernelTraceInstance`: a concrete three-instruction run (`PNEW`, `MORPH_ID`, `MORPH_ASSERT`) from `abs_zero`, with `proves_bits` k meaning $k \leq 8$. Its bound is `kernel_trace_instance_bound` ($k \leq 9$) and it is inhabited (`kernel_trace_instance_inhabited`, $k = 8$).

Chapter 8

Computational Universality and Subsumption

8.1 Strict Containment

Theorem 8.1 (Turing $\subsetneq$ Thiele, as instruction sets (kernel/Subsumption.v)). *The Kernel instruction language is **properly extended** by its sighted programs:*

$$\exists p. \text{sighted}(p) \wedge \neg \text{turing}(p)$$

There exist Kernel programs that are not Turing programs. The witness is the one-instruction program [H_ClaimTapeIsZero 0]. The file stands alone: nothing in it mentions VMState, vm_step, vm_mu, or instruction_cost. The separation is syntactic: the instruction language is larger. It is not a claim that the Thiele model computes any function a Turing machine cannot. The extra structure is new content in what a program can say, not in what it can compute.

Turing completeness of the VM instruction set is a separate result. TuringCompletenessISA.v compiles a 2-counter Minsky machine to five opcodes executed through vm_apply, within the 64-bit word bound. thiele_strictly_extends_turing (ProperSubsumption.v) simulates every Turing computation in a local Thiele_Config record with cost certificates. TuringClassicalEmbedding.v takes the classical opcode fragment as the Turing notion, with the identity embedding into the Thiele programs; it does not simulate KernelTM.v inside that fragment.

8.2 Oracle Cost Lower Bound

Theorem 8.2 (Oracle pricing self-consistency (conceptual; not separately formalized)). *Inside the No Free Insight framework, the universal lower bound on certification cost (universal_nfi_any_substrate in coq/kernel/nfi/UniversalCertificationCost.v) immediately implies that any sound oracle satisfying the one-step rule cs_cert_costs (cost ≥ 1 on any uncertified-to-certified transition) charges ≥ 1 on any trace that takes the oracle's certification from false to true. A threshold of K units is the content of universal_nfi_quantitative (QuantitativeNoFI.v). There is no separate oracle-impossibility file in the codebase; the oracle case is a direct instantiation of the universal theorem at a CertificationSystem whose state encodes oracle responses. The justification for charging $\mu > 0$ on oracle queries comes from NoFI, which is an accounting law, not a physical one: gaining certified information about undecidable propositions has positive μ -cost.*

8.3 μ -budget predicates

Definition 8.1 (μ -budget predicates (kernel/MuComplexity.v)). Let $p_{\text{fuel}}, p_{\mu} : \mathbb{N} \rightarrow \mathbb{N}$ be numeric bounds and $\text{size} : \text{VMState} \rightarrow \mathbb{N}$ a size function. Polynomiality is not required.

- **error_free_preservation_budget** means that every input has an error-free trace within the length and μ bounds, and that f has equal values on the initial and final states whenever μ increases.
- **positive_input_certification_budget** means that every input satisfying f has a trace within those bounds ending with `vm_certified = true`.

Theorem 8.3 (Zero- μ traces satisfy the preservation budget (kernel/MuComplexity.v), (S)). *zero_mu_traces_satisfy_preservation_budget: if every input has a bounded, error-free, zero- μ trace, the preservation budget holds with μ -bound zero for every Boolean state predicate. Its positive- μ antecedent never holds.*

Remark 8.1. These predicates supply neither a uniform solver with a correct output nor a sound verifier rejecting negative inputs. They do not define decision or verification complexity classes. The implication between them is named as a premise, not proved. The factored-search witness in `StructuralAdvantage.v` gives a concrete program-level μ -cost comparison: a program whose instruction costs sum to 18 (`sighted_program_total_cost_is_eighteen`) and whose iteration count is $2N$, against N^2 for the blind program. The VM runs are proved for every N by loop invariants: from `init_state` the blind program on the last of N^2 cells stops with N^2 passes and $\mu = 0$, and the sighted program stops with $2N$ passes and $\mu = 18$; at a price of λ passes per unit of μ the sighted run is cheaper exactly when $2N + 18\lambda < N^2$ (`time_tax_theorem`). That is not a class separation.

Theorem 8.4 (μ -Cost Hierarchy (kernel/MuHierarchyTheorem.v), (S)). *For every $k \geq 1$, the μ -dimension admits a strict hierarchy:*

1. **Achievability:** *there exists a trace from `init_state` with μ -cost exactly k that is level- k certified, the run executes a `CERTIFY` δ instruction with $S(\delta) \geq k$ and ends with `vm_certified = true`.*
2. **Lower bound:** *any level- k certified trace from `init_state` must have spent at least k units of μ -cost.*

Formally (mu_hierarchy_theorem): for every $k \geq 1$, the witness trace $[\text{instr_certify}(k-1)]$ ($\text{cost} = S(k-1) = k$) executes a level- k `CERTIFY` from `init_state`; conversely, the trace-side predicate `level_k_certified` forces $\text{trace_mu_cost} \geq k$ via `level_k_certification_cost_floor`, since the `CERTIFY`'s declared cost appears in the executed-instruction ledger and the ledger sum equals the trace cost (`run_vm_mu_conservation`).

Corollary (`mu_hierarchy_no_upper_bound`): *no fixed μ -budget suffices for all levels. The μ -dimension is unbounded and non-collapsible. Zero Admitted.*

Remark 8.2 (μ -Hierarchy as a P-vs-NP analogue, not a P-vs-NP result). Theorem 8.4 is the μ -analogue of the classical time hierarchy theorem in shape. Its content is lighter. The level of a certificate is defined by the operand δ of the `CERTIFY` that issued it, and the cost schedule charges $\delta + 1$ for that instruction, so the floor comes from the schedule, not from a search or diagonal argument. It is not a statement about classical P versus NP, which this project does not touch. In the certified-structure setting it makes one thing precise: issuing a certificate at level k costs $\mu \geq k$, because the schedule charges $\delta + 1$ for the `CERTIFY` that sets the level. It says nothing about how hard the certified thing was to find. The hierarchy is strict and unbounded, with zero `Admitted`, no oracle, and no diagonalization. The lower-bound proof is

trace-side: a level- k certified run executed a specific CERTIFY δ with $S(\delta) \geq k$, and `executed_instruction_cost_recorded` plus `ledger_sum_contains_lower_bound` place that cost in the ledger sum, which `run_vm_mu_conservation` equates with the trace's μ -cost.

8.4 Intrinsic level hierarchy

Definition 8.2 (Intrinsic level (coq/IntrinsicLevelHierarchy.v)). A final state s is *intrinsically certifiable at level $\geq k$* , written `level_intrinsic_at_least(k, s)`, iff every `run_vm` trace from `init_state` that ends at s with `vm_certified = true` executes at least k cert-setter instructions. Cert-setter here is the positive-cost class `is_cert_setter` (REVEAL, EMIT, LJOIN, LASSERT, READ_PORT, CERTIFY, MORPH_ASSERT, and the CHSH_LASSERT family), not only instructions that write the CSR:

$$\begin{aligned} & \forall \text{fuel trace. } \text{run_vm}(\text{fuel}, \text{trace}, \text{init_state}) = s \\ & \wedge s.\text{vm_certified} = \text{true} \Rightarrow \text{cert_events_in}(\text{fuel}, \text{trace}, \text{init_state}) \geq k. \end{aligned}$$

The level is a property of the *final state*, not of any particular trace reaching it.

Theorem 8.5 (Intrinsic level forces μ (coq/IntrinsicLevelHierarchy.v::level_intrinsic_forces_mu)). *If `level_intrinsic_at_least(k, s)` holds and `s.vm_certified = true`, then for every trace reaching s from `init_state`, `s.vm_mu` $\geq k$. Zero Admitted.*

Theorem 8.6 (Strict separation (coq/IntrinsicLevelHierarchy.v::level_k_plus_1_forces_more_than_k_cert_events)). *For every $k \geq 0$, if `level_intrinsic_at_least(k + 1, s)` holds and `s.vm_certified = true`, then every trace reaching s executes strictly more than k cert-setter instructions. Zero Admitted.*

Remark 8.3 (Relation to Theorem 8.4). Theorem 8.6 is the state-level counterpart of the trace-level μ -hierarchy. The trace-level version (`mu_hierarchy_theorem`, `level_k_certification_cost_floor`) defines level on the trace, a trace is level- k certified iff it executes a CERTIFY δ with $S(\delta) \geq k$. The state-level version defines level on the final state, every trace reaching it requires at least k cert-setter executions. The two are complementary entry points to the same hierarchy: the trace-level theorem gives the achievability witness (`[instr_certify(k-1)]` from `init_state`); the state-level theorem gives the lower bound on μ for any state intrinsically requiring k cert-events. Both theorems are closed under the global context.

8.5 Verifier model and the bare-setting impossibility

Definition 8.3 (Bare verifier (coq/VerifierModel.v)). The bare-setting transcript type is `BareTranscript := list StrictClassicalState`. A bare verification problem is a record `BareVerificationProblem` carrying

- `bare_honest_claim : VMState → Prop`;
- `bare_explains : VMState → BareTranscript → Prop` (the transcript-honestly-witnessed-by-state relation);
- `bare_cheap_budget, bare_recompute_cost : ℕ`.

A bare verifier is a record `BareVerifier` with `bv_decide : BareTranscript → bool` and `bv_cost : BareTranscript → ℕ`.

Definition 8.4 (Soundness, completeness, cheapness (coq/VerifierModel.v)). For a problem P and verifier V :

- $\text{bare_sound}(P, V) := \forall t. V(t) = \text{true} \Rightarrow \forall s. \text{bare_explains}(P, s, t) \Rightarrow \text{bare_honest_claim}(P, s).$
- $\text{bare_complete}(P, V) := \forall s t. \text{bare_honest_claim}(P, s) \Rightarrow \text{bare_explains}(P, s, t) \Rightarrow V(t) = \text{true}.$
- $\text{bare_cheap}(P, V) := \forall t. \text{bv_cost}(V, t) \leq \text{bare_cheap_budget}(P).$

The soundness condition is *strong*: the claim must hold for every state explaining the transcript, not merely for some.

Theorem 8.7 (Bare-setting impossibility (coq/VerifierImpossibility.v::bare_setting_no_sound_complete_verifier)). *There is no bare verifier simultaneously **bare_sound** and **bare_complete** on the verification problem **mu_eq_one_problem** whose claim is $\text{vm_mu}(s) = 1$ and whose **bare_explains** relation holds for the two witness states from coq/NecessityOfMuLedger.v against the shared strict-shadow trace.*

Proof. `po1_state_A` (the cost-1 CERTIFY 0 result, $\mu = 1$) and `po1_state_B` (the cost-0 PNEW [] 0 result, $\mu = 0$) project to identical strict-shadow traces. Completeness applied to *A*’s honest run forces *V* to accept the shared transcript. Strong soundness applied to the same accepted transcript at the explanation by *B* forces $\text{vm_mu}(B) = 1$, contradicting $\text{vm_mu}(B) = 0$. Print Assumptions returns Closed under the global context. $\square$

Theorem 8.8 (Three sufficient interfaces (coq/VerifierEscape_{Substrate,Hardness,Interaction}.v)). *Each kernel theorem below exhibits a constant-cost verifier that simultaneously satisfies soundness and completeness on the μ -sensitive claim by augmenting the transcript with non-classical structure:*

- ***substrate_escape_succeeds**, transcript is `list VMState`; verifier reads μ directly.*
- ***commitment_contract_verifier**, transcript is `BareTranscript` $\times$ `bool` (commitment bit); the `CommitmentBitContract` supplies exact agreement between the bit and the claim for every explaining state. The verifier reads the bit at abstract unit cost. No cryptographic security or runtime bound is proved.*
- ***interactive_escape_succeeds**, transcript is `BareTranscript` $\times$ $\mathbb{N}$ (response); verifier checks the response against the claim.*

Each closes under the global context with zero Admitted.

Theorem 8.9 (Factorisation impossibility (coq/VerifierExhaustiveness.v::V_does_not_factor_through_classical)). *Let T be a transcript type, $\pi : T \rightarrow \text{BareTranscript}$ a projection, and explains_T an explanation relation. Given supplied colliding transcripts $t_A, t_B : T$ such that $\pi(t_A) = \pi(t_B)$, where t_A explains the fixed $\mu = 1$ witness and t_B explains the fixed $\mu = 0$ witness, any verifier sound and complete for the named claim satisfies $\neg \text{factors_classical } \pi V$.*

Remark 8.4 (The verifier corollary). The collision and both explanation premises are hypotheses, not conclusions. The theorem does not construct a collision for arbitrary T or π , and it does not classify all verifier designs. The full-state transcript, exact commitment-bit contract, and reported response above are three sufficient interfaces. The commitment-bit theorem is not a computational-hardness result. The substrate channel is the option the structural axis makes available; hardness and interaction are the two routes classical cryptography and complexity already used, and here the hardness route appears as the exact commitment-bit contract.

Chapter 9

μ -Conservation and Receipt Integrity

9.1 The μ -Ledger Identity

Theorem 9.1 (μ -Ledger Conservation (MuLedgerConservation.v)). *For any bounded execution `run_vm` fuel τ s :*

$$\mu_{\text{final}} = \mu(s) + \sum_{j=0}^k \text{cost}(i_j)$$

where $i_0, \dots, i_k$ are the instructions the run executes (`run_vm_mu_conservation`). The sum is over executed instructions, so it equals the sum over the list τ only for straight-line code; for the list fold `run_instrs` the increment is `trace_total_cost`(τ). Every instruction increases μ by exactly its declared cost. No axioms, no admits.

Corollary 9.2 (Monotonicity). μ never decreases: $\mu(s) \leq \mu(\text{Run}(\tau, s))$ for all τ .

Corollary 9.3 (Irreversibility Bound). `irreversible_bits`(τ) $\leq \mu_{\text{final}} - \mu_{\text{init}}$.

Theorem 9.4 (μ Decomposition (MuLedgerConservation.v)). *Given a per-instruction split $\text{cost}(i) = \text{blind}(i) + \text{sighted}(i)$, supplied as a premise (`mu_component_split`), a bounded run satisfies $\mu_{\text{final}} = \mu(s) + \mu_{\text{blind}} + \mu_{\text{sighted}}$, where each component is summed over the executed instructions (`bounded_run_mu_decomposition`). The theorem does not decide which instructions are blind or sighted.*

9.2 Receipt Integrity

Definition 9.1 (Receipt). A receipt $r = (\text{step}, \text{instr}, \mu_{\text{pre}}, \mu_{\text{post}}, h_{\text{pre}}, h_{\text{post}})$ records one transition.

Theorem 9.5 (Receipt Integrity (ReceiptIntegrity.v)). *A valid receipt chain has final μ equal to $\mu_{\text{init}} + \sum \text{costs}$, the stepwise sum of the chain's instruction costs (`valid_chain_mu_equals_computation`). It does not attest that a run happened. Any receipt claiming $\Delta\mu \neq \text{cost}(\text{instr})$ fails validation.*

Theorem 9.6 (Inconsistent-Increment Rejection (ReceiptIntegrity.v)). *A receipt with $\mu_{\text{post}} = \mu_{\text{pre}} + d$ and $d \neq \text{instruction_mu_delta}(\text{instr})$ fails `receipt_mu_consistent` (`forged_receipt_fails_validation`). This is soundness of the validation predicate, not unforgeability: the state hash is a natural number, and no cryptographic hash or adversary model is defined. Overflow values ($\mu > 2^{31} - 1$) are rejected at the receipt layer.*

Chapter 10

Gauge Shifts of the Ledger and a Noether-Style Invariance

10.1 The Partial $\mathbb{Z}$ -Action (Gauge Group)

Definition 10.1 (Gauge Shift). For $\delta \in \mathbb{Z}$, the gauge shift $\sigma_\delta : \mathcal{S} \rightarrow \mathcal{S}$ maps $S \mapsto S[\mu \leftarrow \text{toNat}(\mu + \delta)]$, preserving all other fields.

Note the truncation. Since μ ranges over $\mathbb{N}$, the Coq definition `z_gauge_shift` computes `Z.to_nat (Z.of_nat vm_mu + delta)`, and `Z.to_nat` clamps negatives to zero. So σ_δ is a total function on states but is *not* injective, and the shifts do not form a $\mathbb{Z}$ -action on all of $\mathcal{S}$.

Theorem 10.1 (Partial Group Action (KernelNoether.v)). *The gauge shifts form a partial $\mathbb{Z}$ -action on states; equivalently, a genuine $\mathbb{Z}$ -action on the sub-poset of states where no intermediate shift underflows:*

1. **Identity:** $\sigma_0(S) = S$. Unconditional (`z_action_identity`).
2. **Composition:** $\sigma_a(\sigma_b(S)) = \sigma_{a+b}(S)$ provided $\mu(S) + b \geq 0$ and $\mu(S) + b + a \geq 0$ (`z_action_composition`, which carries both hypotheses).
3. **Inverse:** $\sigma_{-n}(\sigma_n(S)) = S$ provided $\mu(S) + n \geq 0$ (`z_action_inverse`).

Remark 10.1 (Why composition needs its guard). The guard on clause 2 is not decoration. Take $\mu(S) = 0$, $b = -1$, $a = +1$: σ_b clamps to $\mu = 0$, so $\sigma_a(\sigma_b(S))$ has $\mu = 1$; whereas $\sigma_{a+b}(S) = \sigma_0(S) = S$ has $\mu = 0$. The two sides differ. Composition fails wherever an intermediate shift underflows, which is why the mechanised theorem carries *two* nonnegativity hypotheses: one for the intermediate state, one for the final state. Drop either and the counterexample above goes through.

10.2 Gauge Invariance

Definition 10.2 (Observable). $\text{Obs}(S)$ extracts the partition region structure, ignoring μ .

Theorem 10.2 (Gauge Invariance (KernelNoether.v)). $\text{Obs}(\sigma_\delta(S)) = \text{Obs}(S)$ for every δ and S . It holds by unfolding: no separately named theorem carries it. `Observable_partition` ignores `vm_mu`; this says nothing about every possible observation of the VM.

Theorem 10.3 (Noether-Style Orbit Invariance (coq/kernel/curvature/KernelNoether.v)).

1. **Forward** (`noether_forward`): If every field other than `vm_mu` is equal (the partition observable, graph, registers, memory, CSRs, PC, μ -tensor, error flag, logic accumulator, machine status, witness counters, and certified flag), the two states lie on the same gauge orbit: some δ gives $\sigma_\delta(S_1) = S_2$.

2. **Gauge Invariance:** Shifting μ does not change *Observable_partition*. This is definitional: *z_gauge_shift* only modifies the *vm_mu* field, and *Observable_partition* reads only *vm_graph*. The equality is discharged by unfolding; no named theorem carries it.
3. **Step Equivariance** (*vm_step_orbit_equiv*): For one *vm_step* and δ with $0 \leq \mu(S_1) + \delta$, a step $S_1 \rightarrow S_2$ under instruction i gives a step from $\sigma_\delta(S_1)$ to $\sigma_\delta(S_2)$ under i . Without the positivity premise the equivariance fails ($\mu = 10$, cost 5, $\delta = -12$).

The μ -ledger is a gauge degree of freedom for the partition observable: its absolute value does not enter it, and only differences $\Delta\mu$ carry meaning there. The “Noether” label is a deliberate analogy: the result follows from the record structure (separate fields for μ and *graph*), not from a deep symmetry principle.

Remark 10.2 (Gauge freedom and absolute μ -claims). Gauge freedom has to be squared with the results that assert absolute values, most directly the verifier argument, which needs $\text{vm_mu}(s) = 1$ to be a well-posed claim, and the μ -hierarchy, which needs “cost exactly k ”. If only differences carried meaning, neither statement would have content.

Every absolute μ -claim in this document is *relative to init_state*, where $\mu = 0$ by definition. Read $\text{vm_mu}(s) = 1$ as $\Delta\mu = 1$ along a run from *init_state* to s ; μ -conservation (*run_vm_mu_conservation*) is what licenses the reading, since it makes the final ledger equal to the trace cost.

The scope is worth naming precisely: σ_δ is defined on *arbitrary* states, not on orbits rooted at *init_state*, so the gauge group as formalised is larger than the equivalence the absolute claims need. Nothing in the development quotients by it. The theorems live on *reachable* states, where the base point pins the gauge, and the gauge chapter describes a symmetry of the state *type* rather than of the reachable dynamics. Note also that *vm_step_mu_monotonic* makes μ non-decreasing along runs, which already breaks the $\mathbb{Z}$ -symmetry on trajectories: the dynamics single out a direction the gauge action does not respect.

Theorem 10.4 (μ -Monotonicity (KernelNoether.v)). $\text{vm_step}(S, i, S') \implies \mu(S) \leq \mu(S')$. The μ -ledger never decreases under the dynamics. This is true by construction: instruction costs are natural numbers (≥ 0), so adding them to the ledger cannot decrease it. The proof examines each step constructor and confirms the arithmetic.

Remark 10.3 (Part V Scope: Conditional Results and Named Axioms). The chapters in this part present results conditional on named physical axioms (linearity in the Bloch parameter for the Born rule, the second law for Landauer, and the *A_QM* bridge from NPA PSD to quantum realizability). They are marked **(C)** or **(R)** throughout. The primary unconditional quantum results, the classical CHSH bound, the algebraic Tsirelson bound, and the biconditional *column_contractive_iff_npa_psd*, appear in the chapters that follow the conditional ones: the classical CHSH bound in Chapter 17, the algebraic Tsirelson bound in Chapter 15, and the biconditional in Chapter 18. Those are pure algebra, no physics axioms.

The results in this part are not primary claims of the project. Each is conditional on the named premises, and none claims physics the proofs do not support.

Chapter 11

The Born Rule

Definition 11.1 (Bloch Sphere Probabilities). For a state with purity vector (x, y, z) :

$$P(|0\rangle) = \frac{1+z}{2}, \quad P(|1\rangle) = \frac{1-z}{2}$$

Definition 11.2 (Measurement μ -Cost).

$$\text{Cost}(x, y, z) = \frac{1 - (x^2 + y^2 + z^2)}{2}$$

This is the linear entropy. For pure states ($r^2 = 1$), $\text{cost} = 0$. For mixed states ($r^2 < 1$), $\text{cost} > 0$.

Theorem 11.1 (Uniqueness of the Born Rule among Linear Rules (BornRule.v), **(C)**). *The Born rule $P(|0\rangle) = (1+z)/2$ is the **unique** value of outcome 0 for a probability assignment satisfying:*

1. *Non-negativity on the Bloch ball ($x^2 + y^2 + z^2 \leq 1$) and normalization: $P \geq 0$, $P(|0\rangle) + P(|1\rangle) = 1$.*
2. *Eigenstate consistency: $P(0, 0, 1) = 1$, $P(0, 0, -1) = 0$ for outcome 0.*
3. *Linearity in z on the Bloch ball: $P(x, y, z) = az + b$ for some a, b .*

Proof: Boundary conditions force $a = 1/2$, $b = 1/2$. No other solution exists. Outcome 1 then follows by normalization (`linear_implies_born`, `valid_linear_rule_is_born_with_cost_side_condition`).

Note: The Coq theorem also lists a `measurement_cost_nonnegative` hypothesis, but it is **unused** in the proof. The hypothesis `measurement_cost_nonnegative` P unfolds to “measurement cost ≥ 0 for valid Bloch vectors,” which is directly from the definition true for all states and does not constrain P . The linearity assumption (item 3) does all the work, it assumes the functional form $P = az + b$ and solves a 2-equation system. This is less “deriving the Born rule from μ -accounting” and more “the Born rule is the unique linear rule consistent with eigenstates.”

Chapter 12

Unitarity

Definition 12.1 (Evolution). An evolution E maps Bloch vectors $(x, y, z) \mapsto (x', y', z')$ with associated μ -cost. The cost is a real field of the record. Nothing in the file computes it from the map.

The predicate **radius_preserving** means preservation of squared Bloch radius on the unit ball. The file does not prove that such a map comes from a Hilbert-space operator. Two premises are supplied, not derived: **respects_info_conservation** (radius loss is at most μ) and **purity_nonincreasing** (radius gain is at most μ).

Theorem 12.1 (Zero Cost and Radius Preservation (Unitarity.v), (C)).

1. Under the conservation premise, $\mu = 0 \implies r'^2 \geq r^2$ for inputs in the unit ball (purity cannot decrease at zero cost). Proven as **zero_cost_preserves_purity**.
2. Under the conservation premise, non-unitary evolution (a witness in the unit ball with positive **info_loss**) requires $\mu > 0$. Proven as **nonunitary_requires_mu**: the proof specializes the conservation hypothesis at the witness, then **lra**.
3. Under both premises, $\mu = 0$ implies **radius_preserving**. Proven as **zero_cost_preserves_radius**: the two premises give $r'^2 \geq r^2$ and $r'^2 \leq r^2$.
4. The converse, radius preservation $\implies \mu = 0$, is stated in the file but not proven as a theorem. It appears as the **Definition radius_preserving_zero_cost**. Because μ is a free field of the record, it could not be proved for every evolution.

Theorem 12.2 (A Dissipation Witness Forces $\mu \geq \gamma$ (Unitarity.v), (C)). Assume the conservation premise, a supplied bound **satisfies_lindblad_bound** $E \gamma$ with $\gamma > 0$, and a witness that the pure state $(1, 0, 0)$ loses exactly γ of squared radius. Then $\mu \geq \gamma$ (**lindblad_requires_mu**). In this record, that loss is paid for. The file names the bound after Lindblad dynamics, but it constructs no Lindblad equation. Its “dissipation rate” is the radius loss, with no time derivative.

Theorem 12.3 (The Scalar Ball-and-Trace Contract (Unitarity.v), (R)). Positivity and trace preservation imply **ball_and_trace_preserving** (**ball_trace_contract_from_premises**). The theorem packages the supplied unit-ball and trace conditions. It has no ancillary-system test and establishes no complete positivity in the operator sense.

Chapter 13

Purification

Theorem 13.1 (Two-Point Decomposition of a Bloch State (Purification.v), **(S)**). *The file calls this the purification principle. It proves the spectral fact that a purification would use. It does not construct a purifying state on a larger system. Every mixed state (x, y, z) with $r^2 = x^2 + y^2 + z^2 \leq 1$ admits eigenvalues λ_1, λ_2 with:*

- $\lambda_1 + \lambda_2 = 1, \quad 0 \leq \lambda_i \leq 1.$
- $(\lambda_1 - \lambda_2)^2 = r^2.$
- *Construction:* $\lambda_1 = (1 + r)/2, \lambda_2 = (1 - r)/2.$

Pure states ($r = 1$) need no external reference (deficit = 0). The maximally mixed state ($r = 0$) has deficit = 1.

Chapter 14

No-Cloning

A cloning operation here is a record of four real numbers: input information I , output informations I_1, I_2 , and a cost μ . The chapter's results are arithmetic on those fields under one supplied premise. They use no Hilbert space, channel, or fidelity between states.

Theorem 14.1 (No Perfect Cloning at Zero Declared Cost (NoCloning.v), **(C)**). *Let $I = x^2 + y^2 + z^2$ be the information content (purity). For a cloning operation with outputs of information I_1, I_2 and μ -cost μ , assume the conservation premise (**respects_conservation**, supplied, not derived):*

$$I_1 + I_2 \leq I + \mu$$

*If $I > 0$ and $I_1 = I_2 = I$, then $\mu \neq 0$ (**no_cloning_from_conservation**). For a pure input, $\mu \geq 1$ (**no_cloning_bloch**). **unitary_cannot_clone** links this to the Unitarity chapter. Under that chapter's two premises and $\mu = 0$, one evolution's output copied into both fields cannot satisfy the premise.*

Theorem 14.2 (Approximate Cloning (NoCloning.v), **(C)**). *Under the same premise, with $I > 0$ and $I_i = f_i I$ for scale factors $0 \leq f_i \leq 1$: $f_1 + f_2 \leq 1 + \mu/I$. At zero cost: $f_1 + f_2 \leq 1$. The file calls f_1, f_2 fidelities. They are scale factors on the information field, not the fidelity between quantum states.*

Theorem 14.3 (No Deletion at Zero Declared Cost (NoCloning.v), **(C)**). *Perfect deletion also requires $\mu \geq I$ (dual of no-cloning), under the deletion record's own supplied premise $I_{\text{out}} + \mu \geq I_1 + I_2$, with $I_{\text{out}} = I_1 = I_2 > 0$ (**no_deletion_without_cost**).*

Chapter 15

The Tsirelson Bound

Remark 15.1 (Epistemological Status: **(S)** algebraic, no physics axioms). The upper bound appears in `coq/kernel/quantum/TsirelsonGeneral.v` (`tsirelson_from_row_bounds`). `AlgebraicCoherence.v` supplies a rational coherent witness with $S \geq 28284/10000$ (`tsirelson_rational_lower_witness`). That witness does not attain the irrational optimum. `TsirelsonGeneral.v` derives $S^2 \leq 8$ from the premises $e_{00}^2 + e_{01}^2 \leq 1$ and $e_{10}^2 + e_{11}^2 \leq 1$: a six-square identity gives $S^2 \leq 4 \sum e_i^2$, and the premises give $\sum e_i^2 \leq 2$. `TsirelsonFromAlgebra.v` provides a self-contained algebraic derivation with achievability witness ($e = \pm 1/\sqrt{2}$, giving $S = 2\sqrt{2}$) and a verified rational bound ($\sqrt{8} < 5657/2000$). Zero admits in both files.

Theorem 15.1 (Tsirelson Upper Bound for the Zero-Marginal NPA Model (`TsirelsonQuantumModel.v`, `TsirelsonFromAlgebra.v`), **(S)**). *If a trace's zero-marginal NPA matrix is PSD (equivalently, the trace is slice-coherent, i.e. column-contractive, by `trace_column_contractive_iff_trace_npa_model`), then $|S_{\text{CHSH}}| \leq 2\sqrt{2}$ (`tsirelson_abs_from_npa_psd`).*

Three algebraic statements back it. `TsirelsonFromAlgebra.v` (`tsirelson_squared`) proves $S^2 \leq 8$ from the two row bounds, by linear arithmetic. `coq/kernel/category/AlgebraicCoherence.v` (`algebraically_coherent_tsirelson_general`) proves $S^2 \leq 8$ for rational correlators that are algebraically coherent (four bounds $|E| \leq 1$ and two rational parameters with four nonnegative 3×3 minors), using `psatz` over $\mathbb{Q}$. The column-contractivity route is `state_column_contractive_implies_tsirelson` in `coq/kernel/nfi/MuLedgerQuantumBridge.v`. No physics premises. Pure polynomial arithmetic.

Chapter 16

Witness Insight Taxonomy

Theorem 16.1 (CHSH Trial Is Tier 0 (`coq/kernel/witness/WitnessInsightGeneral.v`), (S)). *chsh_trial_is_tier0: The CHSH_TRIAL instruction never touches either certification channel:*

- *chsh_trial_preserves_cert_addr: CHSH_TRIAL does not change csr_cert_addr.*
- *chsh_trial_preserves_vm_certified: CHSH_TRIAL does not change vm_certified.*

Recording CHSH trial outcomes activates no certification channel, so No Free Insight puts no floor under it. The counters go up; the cert channels are untouched. Its charge is its own μ -delta operand, which may be 0. Zero Admitted.

Theorem 16.2 (Three-Tier Observation Taxonomy, (S)). *In coq/kernel/witness/WitnessInsightGeneral.v, witness_insight_complete_taxonomy is the conjunction of three proved facts, one per tier. The tiers are a reading aid. The theorem does not prove that every instruction falls into exactly one of them, and Tier 2 events are also certified events.*

Tier 0 *Raw observation (can be free): Records data without activating any certification channel. CHSH_TRIAL is the canonical example. Cost: $\Delta\mu = 0$ is possible; no cert channel is touched.*

Tier 1 *Certified structural insight (always costs ≥ 1): An instruction that, at a given state, moves csr_cert_addr from 0 to non-zero or vm_certified from false to true is in this tier. Proven: certified_witness_insight_nonfree shows μ increases by at least 1.*

Tier 2 *Certified non-local witness insight (always costs ≥ 1): A certified execution whose WitnessCount statistics show $S > 2$. nonlocal_witness_insight_nonfree: getting certified with non-local evidence requires $\mu \geq 1$. In the taxonomy theorem this is a trace statement: a run of the certification machine from an uncertified state to a certified state with a certified CHSH violation raises μ by at least 1.*

witness_insight_nonfree_general is the Tier 2 trace statement. Its premises are an uncertified start, a certified end, chsh_violation_certified of the end state, and a run of thiele_cert_machine. The conclusion is that μ increases by at least 1. The Tier 1 instruction statement is certified_insight_nonfree. Of the three tiers, only Tier 0 can be free. Zero Admitted.

Chapter 17

Bell's Theorem in Morphism Coupling Language

Definition 17.1 (Setting-Outcome Coupling (`coq/kernel/quantum/CHSHCouplingBridge.v`)). The `chsh_setting_outcome_coupling` maps a `WitnessCount` record to a coupling (list of $(nat \times nat)$ pairs). Each pair records a (setting-pair-index, outcome-type) observation. A coupling is **separable** (functional) when it maps each setting to at most one outcome type; **non-separable** otherwise.

Theorem 17.1 (Locally Consistent Strategy Gives Separable Coupling (`coq/kernel/quantum/CHSHCouplingBridge.v`), (S)). *locally_consistent_gives_separable_coupling: Any WitnessCount configuration consistent with one deterministic local strategy $(a_0, a_1, b_0, b_1) \in \{0, 1\}^4$ (`WCLocallyConsistent`) gives a separable (functional) coupling. Zero Admitted.*

Theorem 17.2 (Local Coupling Bound (`coq/kernel/quantum/CHSHCouplingBridge.v`), (S)). *locally_consistent_classical_bound: For counts consistent with one deterministic local strategy (a_0, a_1, b_0, b_1) of bits, with every setting pair sampled (`WCLocallyConsistent`), $|S| \leq 2$. This is the classical bound for a single deterministic strategy, stated in the coupling language. It does not cover mixtures or hidden-variable distributions. Zero Admitted.*

Theorem 17.3 (CHSH Violation Rules Out Local Couplings (`coq/kernel/quantum/CHSHCouplingBridge.v`), (S)). *chsh_violation_rules_out_locally_factorizable_coupling: If WitnessCount statistics satisfy $S > 2$, then for every deterministic local strategy of bits (a_0, a_1, b_0, b_1) , the counts are not `WCLocallyConsistent` with it. The exclusion is stated in the coupling language, not just the statistical layer. Zero Admitted.*

Remark 17.1. These results connect the categorical morphism layer directly to Bell nonlocality. A claim that two modules are entangled (via `MORPH_ASSERT` on a non-separable coupling) cannot be made for free: the NoFI law charges at least 1 for the `MORPH_ASSERT`, and the coupling theorem rules out every deterministic local strategy consistent with counts showing $S > 2$. “Entangled” is the reading of a non-separable coupling. The theorems speak about counts and couplings, not about physical systems.

Chapter 18

Slice Coherence and the Zero-Marginal NPA Matrix: The Biconditional

This algebraic result has no project-local axioms or admitted lemmas; its assumption receipt includes standard classical-real and functional-extensionality axioms.

Definition 18.1 (Column Contractivity (Slice Coherence)). **18.1 The CHSH statistic S and its witness counts**

The central quantum object of this part is defined here, since it is used throughout without ever having been given as a function of machine state.

Definition 18.2 (Correlator from witness counts (`CHSHStatisticalBridge.v`)). A VM state carries a `WitnessCounts` record in $\mathbb{N}^8$, holding for each setting pair $(x, y) \in \{0, 1\}^2$ the tallies `samexy` and `diffxy`. Writing $N_{xy} = \text{same}_{xy} + \text{diff}_{xy}$, the empirical correlator is

$$E_{xy} = \begin{cases} \frac{\text{same}_{xy} - \text{diff}_{xy}}{N_{xy}} & N_{xy} > 0, \\ 0 & N_{xy} = 0. \end{cases} \quad (18.1)$$

The $N_{xy} = 0$ branch is a definitional convention (no trials, no evidence), not a claim that uncorrelated is the right default.

Definition 18.3 (CHSH statistic (`chsh_S`, `TsirelsonFromMu.v`)).

$$S = E_{00} + E_{01} + E_{10} - E_{11} \in \mathbb{R}. \quad (18.2)$$

Note the sign on the last term, and note that in `CHSHStatisticalBridge.v` the same statistic is assembled with the $(1, 1)$ arguments reversed, giving $E_{11}^* = (\text{diff}_{11} - \text{same}_{11})/N_{11}$, so that all four terms may be added. The two presentations agree.

The classical bound is $|S| \leq 2$ and the Tsirelson bound is $|S| \leq 2\sqrt{2}$. Both are proved here as *algebraic* facts with no physics axioms. Where the development states the Tsirelson bound in squared form it appears as $S^2 \leq 8$, which is the same bound: $\sqrt{8} = 2\sqrt{2}$.

A correlation tuple $(E_{00}, E_{01}, E_{10}, E_{11})$ is **slice-coherent** (equivalently, column-contractive; the predicate tests slice membership rather than truthfulness, and the name records that) if:

$$1 - E_{00}^2 - E_{10}^2 \geq 0 \quad (\text{condition 1}) \quad (18.3)$$

$$1 - E_{01}^2 - E_{11}^2 \geq 0 \quad (\text{condition 2}) \quad (18.4)$$

$$(1 - E_{00}^2 - E_{10}^2)(1 - E_{01}^2 - E_{11}^2) \geq (E_{00}E_{01} + E_{10}E_{11})^2 \quad (\text{condition 3}) \quad (18.5)$$

These are exactly the PSD conditions on the 5×5 zero-marginal NPA moment matrix.

Theorem 18.1 (Slice Coherence $\iff$ NPA PSD (coq/kernel/quantum/QuantumPartitionPSD.v, corollary column_contractive_iff_npa_psd), (S)). *column_contractive_iff_npa_psd:*

$$\text{slice_coherent}(\mathbf{E}) \iff \text{npa_psd}(\mathbf{E})$$

where *slice_coherent* means the three column-contractivity conditions hold, and *npa_psd* means the 5×5 zero-marginal NPA moment matrix is positive semidefinite.

Forward direction (*zero_marginal_npa_column_contractive_implies_psd* in coq/kernel/nfi/MuLedgerQuantumBridge.v): *column-contractive* $\Rightarrow$ *NPA PSD*. Proved by completing squares: the quadratic form splits into three squares and a 2×2 block that is PSD by the three conditions.

Reverse direction (*npa_psd_implies_column_contractive* in coq/kernel/quantum/QuantumPartitionPSD.v): *NPA PSD* $\Rightarrow$ *column-contractive*. Proved by specializing the PSD condition at the test vectors $(0, -E_{00}, -E_{10}, 1, 0)$, $(0, -E_{01}, -E_{11}, 0, 1)$, and the family $(0, -(E_{00}t + E_{01}), -(E_{10}t + E_{11}), t, 1)$, obtaining a non-negative quadratic in t , then closing via *quadratic_nonneg_discriminant*.

This biconditional has no project-local axioms; its assumption report includes standard-library real-number and functional-extensionality assumptions. No physics assumptions. Pure polynomial arithmetic. The zero-marginal NPA framework is the orthogonal-observables slice of the quantum elliptope ($\rho_{AA} = \rho_{BB} = 0$), not the full elliptope.

Key consequences:

- *PR box* ($E_{00} = E_{01} = E_{10} = 1, E_{11} = -1$) fails condition 1 immediately ($1-1-1 = -1 < 0$). Direct arithmetic; no separate theorem needed.
- *state_column_contractive_implies_tsirelson* (coq/kernel/nfi/MuLedgerQuantumBridge.v) directly proves *slice-coherent* $\Rightarrow |S|^2 \leq 8$ (equivalently $|S| \leq 2\sqrt{2}$); its proof composes *state_column_contractive_implies_npa_gram* with *npa_psd_implies_tsirelson_bound*. The Cauchy-Schwarz inner step (*PSD* $\Rightarrow$ row bounds $\Rightarrow |S|^2 \leq 8$) is *tsirelson_from_row_bounds* in coq/kernel/quantum/TsirelsonGeneral.v. The contrapositive ($|S| > 2\sqrt{2} \Rightarrow$ not slice-coherent) follows immediately. The converse is false and is not claimed.
- *Turing-classical correlations* $(1, 0, 1, 0)$ satisfy $|S| = 2$ but fail condition 1: $1-1-1 = -1 < 0$. A Turing machine can output this correlation; the slice gate rejects it as slice-incoherent (the full-elliptope gate of the next section accepts it, the point is inside the elliptope, outside the zero-marginal slice). Direct arithmetic from *column_contractive_iff_npa_psd*.

Zero Admitted. Zero named hypotheses.

18.2 The Elliptope Completion and Its Gate

The biconditional above characterizes the zero-marginal slice. The full correlator quantum set is characterized in coq/kernel/quantum/ElliptopeCompletion.v as an existential completion: $(E_{00}, E_{01}, E_{10}, E_{11})$ is **elliptope-realizable** (*elliptope_realizable*) iff some assignment of the cross moments $x = \langle A_0 A_1 \rangle$, $y = \langle B_0 B_1 \rangle$ makes the completed 5×5 moment matrix PSD.

Theorem 18.2 (Elliptope completion, kernel-tier). *All of the following hold with Print Assumptions reporting only the stdlib classical-reals and functional-extensionality families:*

- *deterministic_strategy_elliptope*: every deterministic sign strategy is elliptope-realizable, witnessed by its own cross moments (rank-one Gram argument).

- *elliptope_finite_mixture / lhv_mixture_elliptope*: the set is closed under arbitrary finite sub-convex mixtures; every LHV correlator is inside.
- *elliptope_tsirelson*: every member satisfies $S^2 \leq 8$, via Cauchy–Schwarz inside the PSD form (*psd_cauchy_schwarz*); row-norm bounds provably fail on the full set.
- *pr_box_not_elliptope*: the PR box admits no completion.
- *beyond_classical_elliptope*: $S = 12/5 > 2$ is attained inside, so the classical inclusion is strict.

The decidable gate (`coq/kernel/quantum/ElliptopeGate.v`) is the $\mathbb{Z}$ -arithmetic check an opcode can branch on: a fraction-free Sylvester branch over supplied completion buckets (strict interior) and a rational LDL^T -certificate branch reaching singular and boundary completions, with soundness (`elliptope_check_full_sound`) into `elliptope_realizable`, computed acceptances of the $\mu = 0$ tightness witness, $(1, 0, 1, 0)$, and the on-Tsirelson-curve point $(3/5, 4/5, 4/5, -3/5)$ (`gate_accepts_pythagorean_boundary`), and the universal refusal `elliptope_full_gate_never_accepts_pr_box`. Scope: correlator level; the only points beyond any exact integer check are those whose every PSD completion is irrational. The gate is not bound to a runtime opcode.

18.3 The Pointer-Observable Criterion

`coq/kernel/frontier/PointerObservable.v` formalizes the schema behind the metering conjecture. An **ecosystem** (`Ecosystem`) is a state type carrying n observers, each holding a fragment map over states; an event E **redundantly proliferates** (`redundantly_proliferating`) iff every observer’s fragment records E ; and E is the **unique pointer among** a designated list of rival events (`unique_pointer_among`) iff E proliferates and no listed rival does. Uniqueness is relative to the named rivals on purpose, absolute uniqueness is false for free. A toy replicated-ledger instance shows the definitions are satisfiable and discriminating: the certification event is the unique pointer among the toy’s two meterable events (`toy_cert_unique_pointer`), a sanity instance only.

`PointerObservableReductions.v` instantiates the schema at models of the five metering disciplines of the reductions tier: proof-of-stake finality, gas metering, TEE attestation, certificate transparency, and proof-carrying verification. In each model every observer returns the selected flag, and a named rival flag is omitted by construction; all five are one construction with the fields renamed. The metered event is proved the unique pointer among its named rivals in each model (`PoS_model_unique_pointer`, `Gas_model_unique_pointer`, `TEE_model_unique_pointer`, `CT_model_unique_pointer`, `PCC_model_unique_pointer`), and the conjunction is exported as `five_labeled_models_have_selected_pointer`, which closes under the global Coq context, no axioms at all, `stdlib` or otherwise.

Scope: these theorems verify that the five formalized ecosystems have the claimed pointer structure. No protocol semantics, deployment, or security property occurs in their statements. Whether the real disciplines have that observer structure is a modeling claim argued in prose; if it holds, it is evidence for the pointer-observable conjecture, not a proof of it. No theorem derives a forced event from these models.

`RecordProliferationSurvey.v` extends the comparison to twelve named candidate events drawn from the proof-of-stake, gas, attestation, transparency, proof-carrying, MAC, and signature observer maps. The conjunction `twelve_candidate_measurements`, proved as `twelve_candidate_measurements_checked`, states the declared Boolean classifications. In one three-observer ecosystem (`swap_ecosystem`), `swapped_event_is_pointer_checked` proves that `second_event` is the unique pointer among `[first_event]`. The MAC-labelled event is non-proliferating in the specified observer map. Neither the label’s cryptographic significance

nor the observer map’s faithfulness to a deployed system occurs in the theorem statement; the real-system reading is therefore model-dependent.

The proposed route from distributed agreement to permanence is refuted in `EcosystemGame.v`. The predicate `strong_pointer_necessity` assumes at least one observer, agreement, Boolean soundness and completeness of every view, and a common local update function with no observer index or coordinator state. In `toggle_game_refutes_strong_pointer_necessity`, two observers both track the event exactly and both update by negation; consensus and authenticity hold while a true event becomes false. If durable views are added, `durable_consensus_implies_permanence` recovers permanence, but the persistence condition is then an explicit premise. Thus consensus and coordinator-free observation alone do not make permanent commits the only possible proliferating events.

18.3.1 The adversarial search

Five confirmations settle nothing on their own. A criterion loose enough to accept anything accepts five things just as easily. `coq/kernel/frontier/PointerObservableCounterexamples.v` tests the criterion in the corner where it is most likely to break, on systems that do not resemble the five.

The claim put at risk is the strong reading:

(PO-STRONG) Every system that must resist forgery of a commitment has a commitment event that redundantly proliferates.

A counterexample must satisfy three conditions: (C1) it must resist forgery of a commitment, in the sense that its designers treat an unearned commitment as the primary failure mode; (C2) it must be deployed and independently evolved, owing nothing to this development; and (C3) its commitment event must fail `redundantly_proliferating`.

Each candidate has two halves. (C1) and (C2) are facts about a real design and are argued in prose. The Coq half fixes an observer map and computes (C3) for it. The file contains no definition of forgery resistance, EUF-CMA security, deniability, memory safety, metering, or deployment, so the Coq results alone neither refute nor confirm a security claim.

Three candidates argue against (PO-STRONG), using a two-part argument rather than an unconditional machine theorem. For each, the non-proliferating observer map is checked in Coq and its faithfulness to the design is argued in prose:

- **Deniable (designated-verifier) authentication** (`deniable_authentication_model_not_proliferating`). The scheme is engineered so a third party cannot be convinced even given the whole transcript, because the designated verifier could have produced it. Non-transferability is the product requirement.
- **Symmetric-key authentication** (`mac_model_not_proliferating`). Existential unforgeability is the defining security notion of a MAC, and verification requires the shared secret, so no record reaches anyone else.
- **Object capabilities** (`capability_model_not_proliferating`). Unforgeability here is unconditional, enforced by memory safety rather than by a cost or a hardness assumption. Nothing is paid, nothing is logged, and no observer outside the holder can tell.

A public-log control (`public_log_model_proliferating`) checks that the search can return *confirms* and is not an instrument that only reports refutations; its rival effort event also fails to proliferate (`public_log_effort_not_proliferating`), so the control discriminates rather than accepting everything.

If those prose correspondences are accepted, the boundary they trace is not the one (PO-STRONG) guessed at. A MAC and a digital signature both resist forgery; only the signature

proliferates. Deniable authentication and certificate transparency both resist forgery; deniability actively prevents third-party conviction while transparency actively requires it. Object capabilities and proof-carrying code both make unearned authority impossible; the capability confines it to the holder while the proof travels to every consumer. In each pair the proliferating member is the one whose commitment must convince parties who were not present. The surviving candidate criterion is therefore:

(PO-PUBLIC) A system whose commitment must convince third parties who did not witness its creation will proliferate records of the commitment event.

The signature observer model makes the commitment visible in each observer fragment and proves proliferation for that model. Its correspondence to deployed signatures is a modeling judgment. Public availability of a verifier does not imply that every observer retains a record.

The verifier constructions give three sufficient evidence interfaces under their respective premises: a full-state transcript, an exact commitment-bit contract, and a reported response. Neither these examples nor the factorization obstruction proves that only one route can involve metering.

Three statements travel together in loose prose and come apart under these candidates:

- (M1) Forgery resistance forces metering. False,** on the prose argument. Signatures take the hardness route; object capabilities achieve unforgeability unconditionally through memory safety, with no cost, no record, and no assumption.
- (M2) Forgery resistance forces record proliferation. False,** on the same argument. Deniable authentication, symmetric MACs, and capabilities all resist forgery while proliferating nothing, and in the first case that is the design goal.
- (M3) Public commitments force record proliferation. Proposed.** The chosen observer models satisfy it. Actual record proliferation needs independent evidence; public verifiability alone is insufficient.

The metering claim supported here is exact. Under supplied colliding transcripts, no verifier that is sound and complete for $vm_mu = 1$ factors through the classical projection (**V_does_not_factor_through_classical**). The pricing statement is carried by No Free Insight, not by this theorem. It is narrower than “forgery resistance,” and it is the version this development supports.

Chapter 19

Spacetime Analogies: a μ -Pseudometric, Reachability Cones, and Discrete Geometry

19.1 The μ -Metric

Definition 19.1 (μ -Distance (MuGeometry.v)). The Coq formalization defines `mu_distance` as the absolute difference of μ -totals:

$$d_\mu(S_1, S_2) = |\mu_{\text{total}}(S_2) - \mu_{\text{total}}(S_1)|$$

This is a one-dimensional projection, not a minimum over paths.

Theorem 19.1 (Pseudometric Properties (MuGeometry.v)). *d_μ satisfies non-negativity, reflexivity ($d(S, S) = 0$), symmetry, and triangle inequality, all inherited from absolute value on $\mathbb{Z}$. Note: d_μ is a **pseudometric**, not a metric: $d(S_1, S_2) = 0$ does not imply $S_1 = S_2$ (many distinct states share the same μ -total). The “geometry” is a 1D projection onto the μ counter.*

19.2 Causal Cones

Definition 19.2 (μ -Reachability Cone). $C^+(S) = \{S' \mid \exists \tau. S \xrightarrow{\tau} S' \wedge \text{cost}(\tau) \leq \text{Budget}\}$. This budget-bounded set is a definition given in this document only. `SpacetimeEmergence.v` defines the unbudgeted reachability `reaches` and the executed-trace relation `exec_trace`; `causal_cone` (KernelPhysics.v) is a list of module IDs targeted by a trace.

This is a structural analogy to causal cones: the set of reachable states given a finite μ -budget plays a role analogous to a light cone. The analogy should not be over-read: there is no Lorentz invariance, no metric signature, and no boost transformations in the Coq formalization.

19.3 No-Signaling (Data Isolation)

Theorem 19.2 (Observational Locality (SpacetimeEmergence.v)). *In a well-formed partition graph, if an existing module M_B (its ID below `pg_next_id`) is not in the target set of any instruction in a trace, then M_B ’s observable region is unchanged after execution (`exec_trace_no_signaling_outside_cone`).*

Remark 19.1. This is a **data isolation** property: operations targeting module A in the partition graph do not affect the lookup of module B . The proof is structural (list operations on disjoint keys), non-trivial mainly for `instr_psplit`/`instr_pmerge` which restructure the

graph. The label “no-signaling” is an analogy. There is no spatial metric or speed limit between modules.

19.4 Discrete Geometry: A Restricted Angle-Defect Identity

The result is a conditional combinatorial identity with an assigned equilateral angle model. No physical interpretation is needed, but its graph, incidence, and boundary-count hypotheses are essential.

The formal statement is in `coq/kernel/curvature/DiscreteGaussBonnet.v`.

Theorem 19.3 (Restricted Angle-Defect Identity, (S)). *For a partition graph g satisfying `well_formed_triangulated`, assign $\pi/3$ to every triangle corner. The predicate includes $3F = 2I + B$, $\sum_v \deg(v) = 3F$, and $B = 3\chi$, where $\chi = V - E + F$; the edge definitions give $E = I + B$. With $\deg(v)$ counting incident triangles, define*

$$D(g) = \sum_v (2\pi - \deg(v)\pi/3).$$

Then $D(g) = 5\pi\chi(g)$. The definition uses 2π at every vertex, including boundary vertices.

Remark 19.2 (Hypotheses and relation to Gauss–Bonnet). The incidence equations yield $D = 2\pi V - \pi F = 2\pi\chi + \pi B$. The extra hypothesis $B = 3\chi$ supplies the coefficient 5π . It is not a general property of planar triangulations with boundary: a quadrilateral disk divided into two triangles has $B = 4$ and $\chi = 1$ and is excluded. The predicate also does not itself provide a planar embedding or a proof that the graph is a topological disk.

For a triangulated surface with boundary cycles, the usual combined interior-curvature and boundary-turning sum uses 2π at interior vertices and π at boundary vertices. Since boundary vertices and edges then have equal counts, this correction subtracts πB and gives $2\pi\chi$. Establishing the appropriate surface conditions and boundary-vertex definition is required for that statement. The file formalizes the restricted identity only.

Thus the proof establishes a valid identity for its defined quantity under its stated restrictions. It neither derives those restrictions from computation nor establishes a physical curvature law or an angle scale from μ -cost.

19.4.1 Cross-link in MuGravity Vocabulary

Theorem 19.4 (Graph-Laplacian Sum-Zero Identity (`coq/kernel/frontier/MuLaplacianSum.v`), (S)). *For every `VMState` s :*

$$\sum_{m \in \text{pg_modules}(\text{vm_graph } s)} \text{mu_laplacian}(s, m) = 0.$$

Proof by edge-pair antisymmetry. `nat_list_disjoint` is symmetric, hence `modules_adjacent_by_region` is symmetric, hence `edge_weight` is symmetric; `mu_gradient` is antisymmetric by definition; the double sum cancels under the Fubini swap. No well-formedness, no calibration. Standalone graph-theoretic fact.

Theorem 19.5 (Calibrated Total Angle Defect Is Zero (`coq/kernel/frontier/PartitionTopologyCrossLink`), (S)). *For every `VMState` s at universal local calibration ($\forall m, \text{calibration_residual}(s, m) = 0$):*

$$\sum_{m \in \text{pg_modules}(\text{vm_graph } s)} \text{geometric_angle_defect}(s, m) = 0.$$

The proof composes two kernel facts: `calibration_residual_zero_iff` (geometry $\leftrightarrow \mu$ -ledger at every module) and `total_mu_laplacian_zero` (Theorem 19.4). The scalar examples `unit_residual_is_nonzero` and `pi_times_unit_is_nonzero` state that 1 and π are nonzero. They do not supply VM counter-instances establishing independence of the premises.

Remark 19.3 (Why this is not the topology version). The discrete angle-defect identity does not imply $\sum_m \mu_laplacian = 5 \cdot \chi$. That inference would require identifying two distinct angle-defect quantities: `DiscreteGaussBonnet.angle_defect` (degree $\times \pi/3$, equilateral) and `MuGravity.geometric_angle_defect` (distance-weighted, $\pi \cdot d_{bc}/(1 + d_{ab} + d_{ac} + d_{bc})$). They are not equal in general: `strong_bridge_counterexample` (in the same file) is a scalar computation: for every $d \geq 1$ the formulas $2\pi - 6\pi d/(1 + 3d)$ and $2\pi - 6\pi/3$ differ. The cross-link makes no claim about the topology invariant. Discrete Gauss-Bonnet remains a kernel theorem; it is simply not load-bearing for this cross-link.

Non-vacuity fails on reachable states. Adjacency (`modules_adjacent_by_region`) means two module regions meet, and on every state reachable from `init_state` the regions are pairwise disjoint. So no two modules are adjacent, every module has no neighbours and no triangles, and the face-graph triangle count is 0 (`reachable_no_adjacent_modules`); the calibration premise holds on a reachable state exactly when it has no modules (`reachable_calibrated_iff_no_modules`). A well-formed triangulated reachable graph with F faces has no interior edge, $3F$ vertices, $3F$ edges and Euler characteristic F (`reachable_triangulated_isolated`), and one PNEW from `init_state` reaches such a graph with $F = 1$ (`reachable_triangulated_exists`). The geometry theorems are therefore theorems about partition graphs in general; the machine reaches only these degenerate cases. `module_triangles` in `MuGravity.v` requires pairwise-adjacent neighbors, an actual graph triangle, which excludes tree-shaped pseudo-witnesses.

19.5 Four-Dimensional Discrete Riemannian Geometry (Reference Only, Not a Main Claim)

Remark 19.4 (This section is not about spacetime). The metric defined below is read from a per-module 4×4 tensor with natural-number entries, so it is a non-negative matrix at each vertex. The Coq files assert no Lorentz signature: there is no timelike direction, no light cone, no causal structure, and no proper time. Calling this metric “spacetime” misdescribes the object even under a reference-only label.

Read this section as: discrete geometry on a 4-dimensional simplicial complex built from partition-graph data. Every general-relativistic word that appears below (Christoffel, Riemann, Einstein, stress-energy) names the corresponding discrete operator of the Coq files. The vacuum result is a $0 = 0$ consistency check, as the scope warning says.

Remark 19.5 (Scope Warning). The following subsections define discrete differential geometry objects (simplicial complex, metric tensor, Christoffel symbols, Riemann tensor, Einstein tensor) over partition graphs and prove results about the vacuum case $G_{\mu\nu} = 0 = T_{\mu\nu}$. This is a $0 = 0$ result: both sides are zero in the vacuum because the module tensors are uniform and mass is zero everywhere. It is **not** a derivation of Einstein’s field equations from computation. The gravitational constant $G := 1/(8\pi)$ is a unit choice (a definition), not a derived quantity. No physical claim is being made. The geometry section’s main result is the Discrete Gauss-Bonnet theorem (Section 19.4). Everything in this section is marked reference-only and does not appear in the primary claim ledger.

The previous sections treated spacetime as a 1D projection via the μ -metric. The next subsections extend to 4D discrete geometry. None of the results here are part of the core formal claims of the project.

19.5.1 4D Simplicial Complex

Definition 19.3 (4D Simplicial Complex (`FourDSimplicialComplex.v`)). A **4D simplicial complex** $\mathcal{K}$ (`SimplicialComplex4D`) is a bare finite record of lists:

- **Vertices** V : module IDs (`ModuleID`)
- **Edges** E : two vertices and an optional direction label in $\{0, 1, 2, 3\}$ (no label means undirected)
- **Faces, cells, 4-simplices**: three, four, and five vertices

`build_4d_complex_from_graph` extracts these lists from overlapping module regions of a partition graph. The predicate `well_formed_4d_complex` checks only the arity of each list entry. Face-closure (every face of a simplex is also in $\mathcal{K}$), shared incidence, orientation, and manifold structure are not part of the predicate.

19.5.2 Metric Tensor from μ -Costs

Definition 19.4 (Computational Metric (`MetricFromMuCosts.v`, `RiemannTensor4D.v`)). The metric tensor at vertex v is the per-module 4×4 μ -tensor of module v , read as a real matrix:

$$g_{\mu\nu}(v) = \text{module_tensor_entry } s \ v \ (\mu \bmod 4) \ (\nu \bmod 4) \quad (19.1)$$

(`full_metric_at_vertex`, aliased as `metric_component`). Its entries are natural numbers, and different vertices can carry different tensors. `MetricFromMuCosts.v` also defines two scalar quantities from the structural mass $\text{mass}(v) = \text{module_structural_mass}(s, v)$, the length of the module's region plus the length of its axiom list, a natural number. The diagonal metric:

$$\text{metric_at_vertex}(s, v, \mu, \nu) = \begin{cases} \text{mass}(v) & \text{if } \mu \bmod 4 = \nu \bmod 4 \\ 0 & \text{otherwise} \end{cases} \quad (19.2)$$

and the edge length $\ell(s, m_1, m_2) = \text{mass}(m_1) + \text{mass}(m_2)$. When the tensor at v is the isotropic diagonal with entries $\text{mass}(v)$, the two metrics agree (`full_metric_compat_diagonal`).

Remark 19.6 (The inverse metric). A tensor with zero determinant has no inverse, so “raising an index” is not available there; this includes the all-zero tensor of the vacuum configuration. `MetricFromMuCosts.v` defines `inverse_metric_at_vertex` by Cramer's rule. The curvature operators of `RiemannTensor4D.v` do not use it: they take the identity as the inverse metric, a declared convention of the discrete operators.

Remark 19.7. Uniform module tensors (`uniform_module_tensor`: every module carries the same 4×4 tensor) make the metric position-independent, and the Christoffel, Riemann, and Einstein tensors below vanish. Differing structural masses make `metric_at_vertex` position-dependent (`Kernel.EinsteinEquations4D.non_uniform_mass_produces_curvature`). No theorem concludes nonzero curvature.

19.5.3 Discrete Differential Geometry

Definition 19.5 (Discrete Derivative (`RiemannTensor4D.v`)). For a scalar field $f : V \rightarrow \mathbb{R}$ and direction μ :

$$\partial_\mu f(v) = f(w) - f(v) \quad (19.3)$$

where w is the first vertex, in the vertex-list order of $\mathcal{K}$, joined to v by an edge of direction μ or an undirected edge. If there is no such vertex, $\partial_\mu f(v) = 0$. There is no averaging over neighbours.

Definition 19.6 (Christoffel Symbols (`RiemannTensor4D.v`)). The connection coefficients are:

$$\Gamma_{\mu\nu}^\rho(v) = \frac{1}{2} \left(\partial_\mu g_{\nu\rho}(v) + \partial_\nu g_{\mu\rho}(v) - \partial_\rho g_{\mu\nu}(v) \right) \quad (19.4)$$

There is no inverse metric and no sum over a fourth index: the identity inverse is the declared convention of the discrete operators.

Lemma 19.6 (Uniform Module Tensors Have Zero Connection (EinsteinEquations4D.v: flat_spacetime_christoffel_zero_general), (S)). *For uniform module tensors:*

$$\text{uniform_module_tensor } s \implies \Gamma_{\mu\nu}^\rho(v) = 0 \quad (19.5)$$

Proof. Uniform module tensors $\implies$ position-independent metric $\implies \partial_\mu g_{\nu\sigma} = 0 \implies \Gamma_{\mu\nu}^\rho = 0$. $\square$

19.5.4 Curvature Tensors

Definition 19.7 (Riemann Curvature Tensor (RiemannTensor4D.v)). The Riemann curvature tensor measures the failure of parallel transport:

$$\begin{aligned} R_{\sigma\mu\nu}^\rho(v) &= \partial_\mu \Gamma_{\nu\sigma}^\rho(v) - \partial_\nu \Gamma_{\mu\sigma}^\rho(v) \\ &\quad + \sum_\lambda \left(\Gamma_{\mu\lambda}^\rho(v) \Gamma_{\nu\sigma}^\lambda(v) - \Gamma_{\nu\lambda}^\rho(v) \Gamma_{\mu\sigma}^\lambda(v) \right) \end{aligned} \quad (19.6)$$

where λ ranges over the vertices of $\mathcal{K}$ (sc4d_vertices), not over the four coordinate indices.

Definition 19.8 (Ricci Tensor (RiemannTensor4D.v)). Contraction of the Riemann tensor:

$$R_{\mu\nu}(v) = \sum_\rho R_{\mu\rho\nu}^\rho(v) \quad (19.7)$$

where ρ ranges over the vertices of $\mathcal{K}$.

Definition 19.9 (Ricci Scalar (RiemannTensor4D.v)). Complete contraction, with the identity as inverse metric:

$$R(v) = \sum_{\mu,\nu} \delta_{\mu\nu} R_{\mu\nu}(v) \quad (19.8)$$

where μ, ν range over the vertices of $\mathcal{K}$. The variants `local_ricci_tensor_4d` and `local_ricci_scalar_4d` (EinsteinEquations4D.v) contract over the four indices 0, 1, 2, 3 instead.

Definition 19.10 (Einstein Tensor (RiemannTensor4D.v)). The Einstein tensor:

$$G_{\mu\nu}(v) = R_{\mu\nu}(v) - \frac{1}{2} g_{\mu\nu}(v) R(v) \quad (19.9)$$

Lemma 19.7 (Uniform Module Tensors Have Zero Curvature (EinsteinEquations4D.v: flat_spacetime_riemann_zero_general, flat_spacetime_einstein_zero_general), (S)). *For uniform module tensors:*

$$\text{uniform_module_tensor } s \implies R_{\sigma\mu\nu}^\rho(v) = 0 \implies G_{\mu\nu}(v) = 0 \quad (19.10)$$

19.5.5 Stress-Energy Tensor

Remark 19.8 (The components below are named by analogy, not by physics). The three definitions that follow assign the letters T_{00} , T_{0i} , T_{ij} to expressions in `module_structural_mass`. The names are borrowed from the relativistic stress-energy tensor; the objects are not those quantities, and three specific mismatches should be stated plainly rather than left for a reader to discover:

1. T_{00} is not an energy density. It is set equal to `mass(v)`, a dimensionless integer attached to a vertex. An energy density is energy per unit volume; there is no volume element anywhere in this construction, and no conversion constant. Compare Section 21.9: a dimensionless integer ledger cannot produce a dimensional constant, which is exactly the obstruction here.

2. T_{0i} is not a momentum density. Momentum density is ρv^i , mass density times velocity. The definition below is $\partial_i T_{00}$, the spatial *gradient* of the energy-density slot. That is a different object with different index structure and different dimensions; a gradient of a density is not a flux. There is no velocity field in this model for it to be built from.
3. $T_{ij} = \text{mass} \cdot g_{ij}$ **silently fixes an equation of state**. The spatial block is the energy-density slot times the metric. Where $g_{ij} = \delta_{ij}$ it is an isotropic block with pressure equal to energy density, $p = \rho$: a *stiff fluid*, the maximally rigid equation of state consistent with causality in the relativistic setting. Nothing in the model motivates that choice; it is a definitional convenience, and it should not be read as a derived result about the matter content.

None of this affects the vacuum theorem, where every component is 0 because $\text{mass} \equiv 0$. They matter for how the section is read.

Definition 19.11 (T_{00} slot: vertex mass (EinsteinEquations4D.v)). The 00 component is set to the computational mass. Per Remark 19.8 this is not an energy density:

$$T_{00}(v) = \text{mass}(v) \quad (19.11)$$

Definition 19.12 (T_{0i} slot: spatial gradient of T_{00} (EinsteinEquations4D.v)). The mixed components are set to the spatial derivative of the 00 slot. Per Remark 19.8 this is a gradient, *not* a momentum density:

$$T_{0i}(v) = T_{i0}(v) = \partial_i T_{00}(v) \quad (19.12)$$

Definition 19.13 (T_{ij} slot: energy density times the metric (EinsteinEquations4D.v)). The spatial block is the 00 slot times the metric at v . Per Remark 19.8, where $g_{ij} = \delta_{ij}$ this is the stiff-fluid equation of state $p = \rho$; it is imposed here by definition, not derived:

$$T_{ij}(v) = \text{mass}(v) g_{ij}(v) \quad (19.13)$$

The off-diagonal entries T_{ij} , $i \neq j$, vanish when the metric at v is diagonal (`diagonal_metric_at`, `stress_off_diagonal_zero_isotropic`).

Definition 19.14 (Stress-Energy Tensor (EinsteinEquations4D.v)). Combining all components:

$$T_{\mu\nu}(s, \mathcal{K}, v) = \begin{cases} T_{00}(v) & \text{if } \mu = \nu = 0 \\ T_{0i}(v) & \text{if } \mu = 0, \nu = i \text{ or } \mu = i, \nu = 0 \\ T_{ij}(v) & \text{if } \mu = i \neq 0, \nu = j \neq 0 \end{cases} \quad (19.14)$$

19.5.6 Einstein's Field Equations

Definition 19.15 (Gravitational Constant (EinsteinEquations4D.v)). In computational units where the fundamental scale is set by μ -costs:

$$G = \frac{1}{8\pi} \quad (19.15)$$

This makes $8\pi G = 1$, normalizing the coupling in Einstein's equations.

Lemma 19.8 (Curvature from μ -Gradients (EinsteinEquations4D.v: `curvature_from_mu_gradients`), (S)). *For uniform module tensors:*

$$\text{uniform_module_tensor } s \implies G_{\mu\nu}(v) = 0 \quad (19.16)$$

Proof. The proof chain is:

1. Uniform module tensors $\implies$ position-independent metric
2. Position-independent metric $\implies$ zero discrete derivatives $\partial_\mu g_{\nu\sigma} = 0$
3. Zero metric derivatives $\implies$ zero Christoffel symbols $\Gamma_{\mu\nu}^\rho = 0$
4. Zero Christoffel $\implies$ zero Riemann tensor $R_{\sigma\mu\nu}^\rho = 0$
5. Zero Riemann $\implies$ zero Ricci tensor and scalar
6. Zero Ricci $\implies$ zero Einstein tensor $G_{\mu\nu} = 0$

All steps are proven constructively in `EinsteinEquations4D.v`. □

Lemma 19.9 (Vacuum Implies Zero Stress-Energy (`EinsteinEquations4D.v`), **(S)**). *For vacuum states (zero mass everywhere):*

$$\forall w, \text{mass}(w) = 0 \implies T_{\mu\nu}(v) = 0 \quad (19.17)$$

Proof. Direct computation:

- $T_{00} = \text{mass} = 0$
 - $T_{0i} = \partial_i(\text{mass}) = \partial_i(0) = 0$
 - $T_{ij} = \text{mass}(v) g_{ij}(v) = 0 \cdot g_{ij}(v) = 0$
-

Theorem 19.10 (Vacuum Case: Both Sides Are Zero (`coq/kernel/curvature/EinsteinEquations4D.v`), **(R)**). *For any VM state s , 4D simplicial complex $\mathcal{K}$, indices μ, ν , and vertex v (`einstein_equation_vacuum`):*

$$(\forall w, \text{mass}(w) = 0) \wedge \text{uniform_module_tensor } s \implies G_{\mu\nu}(s, \mathcal{K}, v) = 8\pi G T_{\mu\nu}(s, \mathcal{K}, v) \quad (19.18)$$

Status: this is 0 = 0. Both sides equal zero: the left side because the module tensors are uniform, the right side because all masses vanish. The gravitational constant $G := 1/(8\pi)$ is a definition (unit choice), not a derived quantity. This is **not** a derivation of Einstein's field equations. It is a consistency check that the discrete geometric objects reduce correctly when mass is everywhere zero. The non-vacuum case is not proven. This theorem does not appear in the primary claim ledger of the project.

Proof. Step 1: Expand the right-hand side using $G = 1/(8\pi)$:

$$8\pi G T_{\mu\nu} = 8\pi \cdot \frac{1}{8\pi} \cdot T_{\mu\nu} = T_{\mu\nu} \quad (19.19)$$

Step 2: Apply Lemma 19.8 (curvature from μ -gradients):

The uniform-module-tensor premise gives:

$$G_{\mu\nu}(v) = 0 \quad (19.20)$$

Vacuum mass alone does not give it: structural mass (region length plus axiom length) does not determine the module tensors.

Step 3: Apply Lemma 19.9 (stress-energy conservation):

For vacuum:

$$T_{\mu\nu}(v) = 0 \quad (19.21)$$

Conclusion: Both sides equal zero:

$$0 = 0 \quad \checkmark \quad (19.22)$$

The complete proof is formalized in `coq/kernel/curvature/EinsteinEquations4D.v` with:

- No project-local axioms
- Zero admitted lemmas
- The assumption receipt contains the standard-library classical-real and functional-extensionality axioms (the file imports the standard classical-logic and functional-extensionality libraries)

□

Remark 19.9 (Vacuum vs. Non-Vacuum). Theorem 19.10 requires the vacuum hypothesis (mass = 0 everywhere) and uniform module tensors. For non-vacuum states with arbitrary mass distributions, the equation $G_{\mu\nu} = 8\pi GT_{\mu\nu}$ is not proved. In classical general relativity, Einstein's equations are *field equations* that constrain which configurations are physically possible, not universal identities satisfied by all states.

The vacuum construction is an internal consistency result. It does not establish that physical spacetime curvature emerges from computation. A non-vacuum physical derivation requires additional geometric and physical premises with independently justified applicability.

Remark 19.10 (Epistemological Status: **(R)** Consistency Relation). This is a **(R)** result: the vacuum theorem reduces to $0 = 0$ (both sides vanish when all masses are zero and the module tensors are uniform; the gravitational constant is a definition). It establishes that the discrete geometric objects are internally consistent in the zero-mass case. It does *not* establish that spacetime geometry emerges from computation in any physically meaningful sense; that identification requires physical evidence outside the theorem. The non-vacuum case (the full Einstein field equations for arbitrary mass distributions) is not proved and is not a claim of this project.

Remark 19.11. The discrete geometric objects (metric, curvature, Einstein tensor) are defined from computational primitives. In the vacuum case (all masses zero, module tensors uniform) they all reduce to zero. This is a consistency check, not a derivation of spacetime emergence. The claim that spacetime geometry is an emergent property of computation would require more: a non-trivial non-vacuum case, a connection to physical observations, and the named A_QM bridge. None of those is formalised here.

19.5.7 Affine Metric-Scaled Operator: Non-Uniform Boundary Results

The first-neighbor discrete operator refutes the non-uniform diagonal extension of the EFE (`boundary_4simplex_nonuniform_diagonal_refuted_at_1`, `DiscreteSimplicialGeometry.v`). The signed affine metric-scaled operator satisfies the full tensor EFE at vertex 1 and is characterized at the outer vertices. The results below hold for the explicit metric `has_nonuniform_isotropic_metric`: the metric at vertex 1 is $2\delta_{ij}$ and at vertices 0, 2, 3, 4 it is δ_{ij} .

Theorem 19.11 (Affine Off-Diagonal Ricci Zero, **(R)**). *coq/kernel/curvature/AffineEFEClosure.v, affine_off_diagonal_ricci_zero: Under the affine metric-scaled symmetric operator, given `has_nonuniform_isotropic_metric` s on the boundary 4-simplex, off-diagonal Ricci vanishes at vertex 1: $R_{\mu\nu}(v_1) = 0$ for $\mu \neq \nu$. Zero Admitted. Zero Section Variables.*

Theorem 19.12 (Full Tensor EFE at Vertex 1 (`coq/kernel/curvature/AffineEFEClosure.v`), **(R)**). *affine_full_tensor_efe: Given `has_nonuniform_isotropic_metric` s and `module_structural_mass` s $1 = 1$ (unit structural mass at vertex 1), the affine metric-symmetric curved Einstein tensor on `boundary_4simplex` satisfies*

$$G_{\mu\nu}(v_1) = 3 \cdot T_{\mu\nu}(v_1)$$

for all $\mu, \nu \in \{0, 1, 2, 3\}$. Zero Admitted.

Theorem 19.13 (Outer Vertex Characterization, (R)). *In `coq/kernel/curvature/AffineEFEClosure.v`, given `has_nonuniform_isotropic_metric s`, at the outer vertices $v \in \{0, 2, 3, 4\}$ of the boundary 4-simplex, the affine operator produces:*

- *`affine_ricci_offdiag_outer`: Off-diagonal Ricci = $3/32$ (non-zero).*
- *`affine_ricci_diag_outer`: Diagonal Ricci = $-45/32$.*
- *`affine_ricci_scalar_outer`: Ricci scalar = $-45/8$.*
- *`affine_einstein_outer`: Einstein tensor = $45/32$ (diagonal) and $3/32$ (off-diagonal).*
- *`affine_efe_fails_outer_offdiag`: For $\mu \neq \nu$, $G_{\mu\nu} \neq 3T_{\mu\nu}$ at the outer vertices; the off-diagonal Einstein tensor is $3/32$ there (`affine_einstein_outer`).*

The affine factor $8 - 5g_{00}$ is specifically tuned to the center vertex geometry; at outer vertices it produces a non-trivial off-diagonal Einstein tensor. Zero Admitted.

Remark 19.12 (Physical scope). The affine operator closes vertex 1 and characterizes the outer vertices. The claim that the affine factor $8 - 5g_{00}$ is the correct physical discretization requires a connection to a physical substrate. It is an empirical claim, not a Coq theorem.

Chapter 20

Information Causality

Theorem 20.1 (IC- μ Equivalence (InformationCausality.v), (R)). *The file defines two record types (`ICScenario` and `MuScenario`) with opaque `Prop` fields, then defines the relation `ic_mu_equivalent` between them, which literally states the equivalence (equal size parameter, equal cost parameter, equivalent bound propositions). `ic_mu_record_projection` extracts its third conjunct; no proof in the file uses `tauto`. No mutual information, probability distributions, or CHSH values appear anywhere in the formalization.*

Remark 20.1. The intended statement, that Bob’s total information about Alice’s data is bounded by the channel’s μ -capacity, is outside the formalized result. The Coq file packages the proposed equivalence as an explicit hypothesis and derives the stated consequence from that hypothesis. It defines neither entropy nor mutual information and proves no bound from the VM semantics; those physical quantities and their connection to μ are outside the formal scope.

Chapter 21

Thermodynamic Bridge: Conditional Links and Their Limits

21.1 Landauer’s Principle

Theorem 21.1 (Landauer-Shaped Erasure Arithmetic (LandauerDerived.v), (C)).

1. *erasure_irreversible*: Erasing ≥ 1 bit implies fan-in = $2^k > 1$ (arithmetic: $2^k > 1$ for $k \geq 1$).
2. *erasure_decreases_entropy*: Erasure decreases system entropy ($\Delta S < 0$), this is just $\text{output} < \text{input} \Rightarrow \text{output} - \text{input} < 0$.
3. *landauer_information_bound*: For any physical erasure, environment entropy increase $\geq$ bits erased. Here a physical erasure is a *PhysicalErasure* record, which carries this bound as a field (see the caveat).
4. *erasure_additive*: Sequential erasures compose additively.

Caveat on item 3: The *PhysicalErasure* record has *second_law_satisfied* as a *field*, not a derived property. The theorem *landauer_information_bound* simply extracts this field, making the bound an input assumption rather than a derivation. The physical energy bound $Q \geq k_B T \ln 2 \cdot n$ is **explicitly not proven** (the file itself notes the conversion factor is not formalized). Also, the *mu* in this file is a standalone synonym for *bits_erased*, it has no formal connection to *vm_mu* in *VMState.v*.

21.2 Two-State Calorimeter Protocol

CalorimeterProtocol.v specifies a microscopic register with ground energy 0, excited energy Δ , excited-state population p , and the discrete master equation

$$p' = p + dt(k_{01}(1 - p) - k_{10}p).$$

At fixed Hamiltonian its bath heat is $Q_{\text{bath}} = \Delta(p - p')$. The canonical reset starts at $p = 1/2$, ends at $p' = 0$, and uses $dt = 1$, $k_{01} = 0$, and $k_{10} = 1$. The theorems *canonical_reset_satisfies_master_equation* and *canonical_reset_heat_exact* prove the update and $Q_{\text{bath}} = \Delta/2$ exactly.

Setting $\Delta = 2k_B T \ln 2$ yields $Q_{\text{bath}} = k_B T \ln 2$ (*selected_gap_gives_landauer_heat*), but this equality selects the energy scale rather than deriving it. With identical populations, rates, and one logical μ event, given positive k_B and T , $\Delta = k_B T \ln 2$ gives half

the Landauer value (`smaller_gap_refutes_unconditional_landauer_floor`); more generally, `master_equation_does_not_fix_heat_scale` holds the dynamics fixed while changing the heat. The protocol is therefore a closed calorimeter blueprint, not an unconditional calibration. Applying a Landauer floor requires an additional thermal-admissibility condition such as local detailed balance or the second law, plus a measured correspondence between the device reset and the VM event. On the VM side the protocol’s count of one logical μ event equals the minimal certification charge (no theorem maps the protocol’s states to VM states): `CERTIFY 0` from `abs_zero` certifies and raises `vm_mu` by `canonical_reset_mu = 1` (`vm_minimal_certification_charges_canonical_reset_mu`), and, given an uncertified state, no VM step switches `vm_certified` on for less (`vm_certification_charges_at_least_canonical_reset_mu`). The correspondence between the device and that event is not proved.

21.3 A2 from Landauer (Factored Form): a Conditional Whose Premises Fail on the Full ISA

Theorem 21.2 (Landauer Strong Form: A2 from Two Landauer-Style Premises (`coq/kernel/frontier/LandauerDissipationStrongForm.v`), (C)). *Given an abstract dissipation function $\phi : \text{vm_instruction} \rightarrow \mathbb{N}$ (interpreted as dissipation per step in Landauer quanta), if*

1. *Landauer’s principle holds in dissipation vocabulary, every class-collapsing step on any bool macro-property has $\phi(i) \geq 1$, and*
2. *the named bridge `cost_dissipation_calibrated` holds, $\text{instruction_cost}(i) \geq \phi(i)$ for every i ,*

then on every cert-flip step $\text{instruction_cost}(i) \geq 1$. The Landauer-style premise mentions only class-collapse and dissipation; `instruction_cost` appears only in the calibration. This is a valid conditional implication, but its two premises are incompatible on the full VM instruction set, as the following remark explains.

Remark 21.1 (Applicability failure of the full-ISA physical premises). `landauer_dissipation_premises_incompatible` proves that no dissipation function satisfies both hypotheses. The instruction `JUMP(1,0)` has zero cost and sends every state’s program counter to 1. For the Boolean macro-property “PC equals 1,” it meets `step_collapses_bool_classes`: a state with PC 0 enters the true class, and states already in that class stay there. Hypothesis (1) therefore requires dissipation at least 1, while hypothesis (2) bounds it by 0.

Thus this theorem does not supply an applicable physical derivation of A2 for the VM. A2 follows directly from its instruction semantics, without either physical premise. A physical account must justify a domain of operations and macro-properties, reconcile its dissipation model with the cost schedule, and exhibit jointly satisfiable premises. Merely losing a distinction under a selected macro-observation does not itself establish physical information erasure.

Remark 21.2 (Integrality of ϕ is a physical assumption, not a typing choice). ϕ is declared with codomain $\mathbb{N}$, and the Landauer premise is stated as $\phi(i) \geq 1$. The integrality is doing real work: it is what makes the floor come out as “at least one” rather than “at least $k_B T \ln 2$ ”. But Landauer’s principle gives a *real-valued* lower bound on dissipated energy per bit erased; nothing in thermodynamics says dissipation arrives in integer multiples of a quantum. Choosing $\mathbb{N}$ as the codomain therefore imports a quantisation assumption through the type signature, where it is easy to miss. It should be read as a named modelling assumption on the same footing as `cost_dissipation_calibrated`, not as a neutral formalisation detail.

Theorem 21.3 (Calibration consistency and retained history, (S)). *For eligible operations $E(i)$ and a schedule $c(i)$, a function d with $E(i) \Rightarrow d(i) \geq 1$ and $d(i) \leq c(i)$ exists exactly when all*

eligible operations have positive scheduled cost. The witness in the converse is $d = c$. A Boolean identity/reset model realizes the conditions with charges zero and one respectively.

For any deterministic step function, the extension

$$(s, h) \mapsto (\text{step}(s, i), s :: h)$$

is injective for fixed i , has a left inverse on its image, and projects to the original transition. In particular, the zero-cost VM jump has an injective history lift. These results establish consistency of an abstract restricted model and failure of macro-collapse as a general erasure test. They do not identify the cost units with heat.

21.4 Permanent Certification Forces a Merge

The history lift escapes the merge by keeping unbounded history. On a finite state space with a certificate that no step revokes, that escape is closed for the certifying step.

Theorem 21.4 (Permanent flip is not injective (`coq/kernel/nfi/PermanentCertification.v`), (S)). *Let S have a duplicate-free enumeration, let $\text{step} : S \rightarrow I \rightarrow S$ and $\text{cert} : S \rightarrow \text{bool}$ be arbitrary, and suppose $\text{cert}(s) = \text{true} \Rightarrow \text{cert}(\text{step}(s, i)) = \text{true}$ for all s, i . If $\text{cert}(s) = \text{false}$ and $\text{cert}(\text{step}(s, i)) = \text{true}$, then $\text{step}(\cdot, i)$ is not injective (`permanent_flip_is_not_injective`). If k distinct uncertified states are switched on by one instruction, that instruction's step has at least k fewer distinct images than inputs on the certified states together with those k (`permanent_flips_collapse_at_least`).*

Theorem 21.5 (A2 from a merging price (`PermanentCertification.v`), (C)). *Under the same premises, if every instruction whose step is not injective costs at least 1, then every certification flip costs at least 1 (`a2_from_merging_price_and_permanence`). The system is then a `CertificationSystem` with its A2 field discharged (`certification_system_from_merging_price`), and the trace floor follows (`permanent_certification_trace_floor`). Honest erasure accounting, whose flag reports every merging instruction, gives the same floor (`honest_erasure_accounting_implies_a2`); the zero-cost rival of `CommitmentVsErasure.v` is not honest in this sense (`commit_without_erasure_system_is_not_honest`).*

Remark 21.3 (Scope and necessity of each premise). The merging price is a named premise standing for Landauer's principle; it is conditional in the sense of (C). The premises are jointly satisfiable (`priced_reset_satisfies_premises`). Each is needed: an unbounded history keeps every step injective while a permanent certificate switches on (`unbounded_history_escapes`); negation switches a revocable certificate on bijectively (`revocable_certificate_escapes`); an unpriced reset certifies at cost zero (`free_merge_escapes`). The 51-opcode VM is not an instance, by design: its ledger is unbounded, and its schedule prices the certifying merge while JUMP merges program counters at zero declared cost (`vm_prices_certifying_merge_leaves_others_free`, below). A finite fragment of it is an instance (`FiniteCertMachine.v`, below). All ten theorems are closed under the global context.

Remark 21.4 (Prior results used). The ingredients are standard: an unconditional set of a bit is Landauer's example of a logically irreversible operation (1961); finite memory forcing erasure is Bennett's observation on reversible computation; the core counting is the pigeonhole principle; the entropy support bound and the monotonicity of entropy under deterministic maps are Shannon's; the dependence of Landauer's bound on the input distribution is the generalized form of Maroney (2009) and Sagawa (2014). The contribution recorded here is the combination about permanent certificates, the merge-or-revoke characterization, and the mechanized statements. Shadow pricing (Section 21.5) is fiber constancy (`decoding_requires_fiber_constancy`) applied to prices.

Theorem 21.6 (Cost of a squeeze (coq/kernel/nfi/PermanentRecordPricing.v), (C)). Assume the counting premise *compression_priced*: for every instruction i and duplicate-free list D , $|D| \leq 2^{\text{cost}(i)} \cdot |\text{nodup}(\text{step}(\cdot, i)[D])|$. On a finite state space with a permanent certificate and m certified states, if instruction i switches k distinct uncertified states on, then $m + k \leq 2^{\text{cost}(i)} m$ (*permanent_flips_compression_bound*) and $\lceil \log_2 \rceil(m + k) \leq \text{cost}(i) + \lceil \log_2 \rceil m$ (*permanent_flips_log_bound*). With $k = 1$ this yields A2 (*a2_from_compression_price_and_permanence*). A four-state stamp attains the bound with cost 2 for $k = 3$, $m = 1$ (*quad_stamp_needs_two*, *quad_stamp_cost_two_is_priced*).

Theorem 21.7 (Forced price and the merge-or-revoke dichotomy (PermanentRecordPricing.v), (S)). Call i forced to be priced when every cost assignment satisfying *merging_steps_priced* charges i at least 1. With decidable equality on instructions, i is forced to be priced iff $\text{step}(\cdot, i)$ is not injective (*forced_priced_iff_merges*). On a finite state space, if i switches a reading on at some state, then either $\text{step}(\cdot, i)$ is not injective or i switches the reading off at some certified state (*flip_merges_or_revokes*, proved constructively by a finite search). Hence an instruction that never revokes the reading and flips it somewhere is forced to be priced (*permanent_record_write_is_forced_priced*). The converse fails: a three-state instruction flips, revokes, and merges (*forced_price_without_permanent_record*). The trace-level floor follows through *universal_nfi_any_substrate* (*compression_priced_trace_floor*). All twelve theorems and corollaries in the file are closed under the global context.

Theorem 21.8 (Entropy form (coq/kernel/nfi/PermanentCertificationEntropy.v), (S) for the bound, (C) for heat). Let $H(p) = -\sum_s p(s) \log_2 p(s)$ over the enumeration, with $0 \log 0 = 0$, and let f_*p be the pushforward of p along $\text{step}(\cdot, i)$. Entropy is at most $\log_2$ of the size of any list covering the support (*entropy_le_log_support*, from $\ln y \leq y - 1$). On a finite state space with a permanent certificate, if i switches $k \geq 1$ distinct uncertified states on beside m certified ones and p is supported on those $m + k$ states, then $H(f_*p) \leq \log_2 m$ (*permanent_step_entropy_ceiling*); for p uniform on them, $H(p) - H(f_*p) \geq \log_2((m + k)/m)$ (*permanent_flip_uniform_entropy_drop*). An injective step leaves H unchanged (*step_entropy_invariant_if_injective*). A whole-unit cost with $\text{cost}(i) \geq H(p) - H(f_*p)$ for every distribution p satisfies A2 (*a2_from_entropy_price_and_permanence*), with the trace floor through *universal_nfi_any_substrate* (*entropy_priced_trace_floor*). Under the named premise *landauer_heat* ($\text{heat} \geq k_B T \ln 2 \cdot \Delta H$), the uniform flip dissipates at least $k_B T \ln((m + k)/m)$ (*permanent_flip_heat_floor*), which is positive for $T > 0$ (*permanent_flip_heat_positive*). For any p and any deterministic f , $H(p) - H(f_*p) = \sum_{p(x) > 0} p(x) (\log_2(f_*p)(fx) - \log_2 p(x))$ (*entropy_drop_as_point_sum*), each term is nonnegative (*entropy_drop_nonneg*), and the sum is positive when $x \neq x'$, $fx = fx'$, $p(x) > 0$, $p(x') > 0$ (*entropy_drop_pos_of_support_merge*). Hence a permanent flip removes entropy for every p positive on the certified states and the flipping state (*permanent_flip_full_support_entropy_drop_positive*), with positive heat under the premise (*permanent_flip_full_support_heat_positive*). For a point mass the drop is 0 and the premise is equivalent to $\text{heat} \geq 0$ (*known_state_step_removes_no_entropy*, *known_state_flip_forces_no_heat*): the forced heat is a worst-case statement over the state distribution, as in the generalized Landauer bound. The file works over Coq's real numbers; its results depend on the standard-library constants *sig_forall_dec*, *sig_not_dec*, *functional_extensionality_dep*, and *classic*, and on nothing project-local.

Theorem 21.9 (A finite instance run by the VM (coq/kernel/nfi/FiniteCertMachine.v), (S)). Let $\text{FState} = \{L_0, L_1, L_2, L_3\} \times \text{bool}$ and $\text{FInstr} = \{\text{FCertify}, \text{FJump } a, \text{FNext}\}$ with $\text{FCertify}(p, c) = (p + 1, \text{true})$, $\text{FJump } a(p, c) = (a, c)$, $\text{FNext}(p, c) = (p + 1, c)$ (slots modulo 4), certificate π_2 , and costs 1, 2, 0. The eight states form a finite enumeration (*fin_finite*), the certificate is permanent (*fin_permanent*), *FNext* is injective and the other two are not, the costs satisfy *merging_steps_priced* (*fin_merging_priced*) and *compression_priced* (*fin_compression_priced*, via the fiber lemma *fiber_bound_compression*), and A2

follows (*fin_a2_from_merging_price*, *fin_a2_from_compression_price*). With window $w(s) = (\text{slot}(\text{vm_pc} \bmod 4), \text{vm_certified})$ and $\text{FCertify} \mapsto \text{CERTIFY } 0$, $\text{FJump } a \mapsto \text{JUMP } a \ 2$, $\text{FNext} \mapsto \text{CHECKPOINT "" } 0$: $w(\text{vm_apply}(s, \iota)) = \text{fstep}(w(s), \iota)$ and μ rises by exactly $\text{fcost}(\iota)$ (*vm_runs_finite_machine*, *vm_pays_finite_price*), for whole runs as well (*vm_runs_finite_trace*); a run of the fragment from uncertified to certified raises μ by at least 1 (*vm_fragment_certification_paid*). The JUMP cost of 2 is a program's choice of μ_delta . All twenty-three theorems and lemmas in the file are closed under the global context.

Theorem 21.10 (What the full VM prices (*FiniteCertMachine.v*), (S)). $\text{vm_apply}(\cdot, \text{CERTIFY } d)$ is not injective: states differing only in *vm_certified* have the same image (*vm_certify_merges*). Every s, i with $\text{vm_certified}(s) = \text{false}$ and $\text{vm_certified}(\text{vm_apply}(s, i)) = \text{true}$ has $\text{vm_apply}(\cdot, i)$ non-injective and $\text{instruction_cost}(i) \geq 1$ (*vm_certifying_step_is_priced_merge*). $\text{vm_apply}(\cdot, \text{JUMP } 0 \ 0)$ is not injective and costs 0 (*vm_jump_is_free_merge*). Hence $\neg \text{merging_steps_priced}(\text{vm_apply}, \text{instruction_cost})$ (*vm_prices_certifying_merge_leaves_others_free*). Thus every certification flip is a priced merge and at least one other merge is free; the theorem does not classify every non-certifying merge. A2 holds for the VM independently (*no_free_certification_certified*).

21.5 Exact Pricing Cannot Be Computed From a Window

Theorem 21.11 (No exact shadow price (*coq/kernel/nfi/ShadowPricing.v*), (S)). Let $\text{step} : S \rightarrow I \rightarrow S$, $\text{cert} : S \rightarrow \text{bool}$, and $\text{obs} : S \rightarrow O$ be arbitrary. A shadow price is $c(s, i) = \text{price}(\text{obs}(s), \text{obs}(\text{step}(s, i)))$. Call a collision two pairs (s_1, i_1) , (s_2, i_2) with equal observations before and after, where the first flips cert from false to true and the second does not. If a collision exists, every shadow price meeting the floor charges a non-flipping step (*shadow_floor_overcharges*), so no shadow price both meets the floor and never overcharges (*shadow_cannot_price_exactly*). The step price $[\text{flip}]$ is exact (*step_price_is_exact*). If $\text{cert} = r \circ \text{obs}$ for some r , an exact shadow price exists (*window_showing_reading_prices_exactly*) and no collision exists (*window_showing_reading_has_no_collision*). On the VM, $(\text{abs_zero}, \text{CERTIFY } 0)$ and $(\text{abs_zero}, \text{JUMP } 1 \ 0)$ collide for *bare_observable*, and $(\text{abs_zero}, \text{CERTIFY } 0)$ and $(\text{abs_zero}, \text{JUMP } 1 \ 1)$ collide for *forget* (*vm_bare_observable_collision*, *vm_forget_collision*), so neither window admits an exact price for VM certification (*vm_bare_shadow_cannot_price_exactly*, *vm_forget_shadow_cannot_price_exactly*). All nine theorems are closed under the global context.

21.6 Which Narrowing a Merge Price Charges

Theorem 21.12 (The machine's spread is priced (*coq/kernel/nfi/KnowledgeNarrowing.v*), (S)). Let $\text{step} : S \rightarrow I \rightarrow S$ with decidable equality on S , and let cost meet *compression_priced*. For every duplicate-free list D and program t , with D' the duplicate-free list of states $\text{run}(s, t)$ for $s \in D$,

$$\lceil \log_2 |D| \rceil \leq \text{cost}(t) + \lceil \log_2 |D'| \rceil$$

(*run_narrowing_priced_log*). No decision tree or payment premise is supplied.

Theorem 21.13 (An observer's knowledge is not priced (*KnowledgeNarrowing.v*), (S)). Let the observer's knowledge after t from s_0 be the states of Ω whose observation along the run equals that of s_0 . On $S = \text{bool} \times \text{bool}$ with $\text{Measure}(x, d) = (x, d \oplus x)$ at cost 0 and $\text{Wipe}(x, d) = (x, \text{false})$ at cost 1, this particular cost meets *compression_priced*, and from $\Omega = \{(0, 0), (1, 0)\}$ the program $[\text{Measure}]$ at cost 0 leaves the display observer one candidate. Hence $\lceil \log_2 |\Omega| \rceil - \lceil \log_2 |\Omega'| \rceil \leq \text{cost}(t)$ fails for this admissible cost (*observer_narrowing_can_be_free*). Since *compression_priced* is only a lower bound, it may also admit costs that overcharge the injective

measurement. The machine's spread stays two (`demon_machine_spread_kept`), and every squeeze-priced cost charges `Wipe` at least 1 (`wipe_costs_at_least_one`). On the VM, `XOR_ADD 2 1 0` separates two states that differ in register 1 through register 2 at cost 0 (`vm_observer_narrowing_at_zero_cost`). All closed under the global context.

21.7 A Recursion Theorem for L

Theorem 21.14 (Kleene and Rice for L (`coq/kernel/foundation/LRecursion.v`), (S)). *L is the weak call-by-value lambda calculus with de Bruijn terms and deterministic left-to-right reduction; $\ulcorner t \urcorner$ is the Scott encoding. There is a closed Q with $Q \ulcorner t \urcorner \rightarrow^* \ulcorner \ulcorner t \urcorner \urcorner$ (Q_spec). For every closed value s there is a closed t with $t \rightarrow^* s \ulcorner t \urcorner$ ($second_recursion$). Every transformer computed on codes by a closed value has an extensional fixed point ($L_recursion_theorem$). No closed term decides, on codes, a property extensional in reached values that holds for one closed program and fails for another (L_rice); in particular halting ($L_halting_undecidable$). All closed under the global context. L is not a `Substrate` instance, since the interface requires a total `run`; the diagonal is restated for L.*

21.8 No Recursion Theorem for the Full VM

Theorem 21.15 (The full-VM recursion premise is false (`coq/kernel/foundation/VMBoundedDecidability.v`), (S)). *Let $run(p, s)$ be the state after 1000 steps of program p from s . It is false that every map f on programs has a q with $run(q, s) = run(f(q), s)$ for every s ($vm_full_recursion_premise_refuted$). For every correct decider d of the bounded shortcut property, the map that sends p to the empty program when $d(p)$ holds and to a fixed shortcut-admitting program otherwise has no such q ($vm_correct_flip_has_no_fixed_point$).*

The bounded shortcut property compares two thousand-step runs from `init_state`, and full-state equality is decidable, so the property has a correct decider; the cutoff, not the size of memory, is what defeats the premise. The conditional diagonal for the VM, `vm_structural_shortcut_undecidable_encoded`, applies only to classes of maps that leave every such flip out. The guest recursion theorem (`vm_guest_recursion_theorem_closed`) is a different statement, under the unbounded sibling's runs and up to final registers and ledger.

21.9 The Dimensional Gap: Why Ledgers Cannot Fix Constants

Theorem 21.16 (Dimensional Gap (`coq/kernel/thermodynamic/DimensionalGapTheorem.v`), (S)). *Let $State$ be any type, $L : State \rightarrow \mathbb{N}$ any dimensionless integer ledger, $\omega : State \rightarrow \mathbb{N}$ any microstate-count function, and $S_{cand} : State \rightarrow \mathbb{R}$ any candidate entropy. Suppose for some $base \geq 2$ and some state s with $L(s) \geq 1$:*

1. **ledger-exponential:** $\omega(s') = base^{L(s')}$ for all s' ;
2. **ledger-linear:** $S_{cand}(s') = \alpha \cdot L(s')$ for all s' ;
3. **Boltzmann-form:** $S_{cand}(s') = k_B \ln \omega(s')$ whenever $\omega(s') > 0$.

Then α is forced:

$$\alpha = k_B \cdot \ln(base). \quad (21.1)$$

This is the limit on deriving physics from computation in this framework, and it should be read as a **no-go**, not as a bridge. The theorem quantifies over *any* type, *any* integer ledger, and *any* exponential microstate count. It makes no reference to any Thiele machine at all. It proves $\alpha = k_B \ln(base)$ from the three stated premises. The reading of the file is that the

unit-conversion constant k_B (an empirical fact about thermodynamics) and the counting base (a structural fact about the substrate’s degrees of freedom) are supplied from outside. Fix both and α follows. The ledger alone does not prefer a value of α , and choosing among the consistent (α, k_B) pairs is the modelling input that the ledger cannot supply. That reading is not itself a Coq theorem.

Two consequences worth stating explicitly:

- The $k_B \ln 2$ appearing in the Thiele instantiation (`SecondLawBoltzmannWall.v`, base = 2 for binary cert-flips) is obtained after supplying k_B and the base from outside: the file proves $\alpha = k_B \ln 2$ for a base-2 candidate under the Boltzmann premise.
- Any claim of the form “this computational framework predicts a physical constant” is blocked by Theorem 21.16 unless it also says which two external scalars it assumed. This applies to the this development as much as to any other.

Read this section before the conditional physics chapters, not after: it is the constraint they operate under.

Chapter 22

Self-Reference and the Thiele Manifold

Remark 22.1 (Scope: a definitional tower). Both files in this chapter stand alone. They import no kernel module and do not mention `VMState` or `vm_mu`. A `System` is a record of a natural number called `dimension` and a predicate on propositions. The meta-system is constructed with dimension one higher. The number 4, the name “spacetime”, and the `mu_cost` of a projection (a difference of two `dimension` fields) are chosen by definition. The chapter records a picture of self-reference escalating through levels and shows that the picture is consistent. It derives nothing about physical dimensions.

22.1 Self-Reference Escalation

Theorem 22.1 (Gödel-Style Escalation (`SelfReference.v`)). *For every self-referential system S there exists a self-referential meta-system Meta that reasons about S and has strictly more dimensions (in the record’s `dimension` field), `self_reference_requires_metalevel`. The witness is `meta_system` S , defined with dimension $\dim(S) + 1$:*

$$\text{self_ref}(S) \implies \exists \text{Meta} : \dim(\text{Meta}) > \dim(S)$$

This is one step. Iterating it gives the tower of `ThieleManifold.v`.

22.2 The Thiele Manifold

Definition 22.1 (Thiele Manifold (`ThieleManifold.v`)). An infinite tower of systems $\{L_n\}_{n \in \mathbb{N}}$ with:

- $\dim(L_n) = 4 + n$ (level n has dimension $4 + n$) in the canonical instance `canonical_manifold`. The record itself requires only $\dim(L_0) \geq 4$ and strict growth.
- $\dim(L_{n+1}) > \dim(L_n)$ (strict enrichment).
- Each level reasons about the level below.
- Self-reference at level n is answered by level $n + 1$.

Theorem 22.2 (Projection to Spacetime (`ThieleManifold.v`)). *The projection π_4 collapses the tower to dimension 4 (spacetime). For $n > 0$:*

- π_4 is lossy: $\dim(L_n) > 4$.

- μ -cost of projection > 0 (*mu_cost_positive_for_projection*); it is the dimension difference $\dim(L_n) - 4$, which is n in the canonical instance (a computation from *canonical_level*'s dimension $4 + n$). This *mu_cost* is the file's own dimension difference, not the VM ledger.

*Spacetime is the “shadow” of the full manifold. That is the file's reading of **pi4** (named **spacetime_shadow**), not a theorem about physical spacetime.*

Chapter 23

Three-Layer Observable Agreement

Remark 23.1 (Agreement, not isomorphism). No isomorphism is constructed in this chapter, and none is claimed: there is no pair of mutually inverse maps between the Coq, OCaml/Python, and RTL layers, and no proof that either composite is the identity. What is established is that any two implementations satisfying a shared observable-projection contract agree on the projected fields after a shared trace: transitivity of equality through a common projection. The Coq layer satisfies the contract (`coq_full_wire_spec`). That the Python and RTL artifacts satisfy it is a premise, not a Coq result.

The distinction is not pedantic. Agreement on a projection is compatible with the layers differing arbitrarily on everything the projection forgets; an isomorphism would forbid that. Since the projection deliberately forgets internal representation, agreement is the right property to want here. Isomorphism would be a stronger claim than the construction supports.

Theorem 23.1 (Observable Comparison Contract (`ThreeLayerIsomorphism.v`)). *The file states an observable-state contract used for cross-layer comparison. An implementation satisfies `FullWireSpec` when each of twelve projected output fields of one step equals the corresponding field of `vm_apply` applied to the projected input. Any two implementations satisfying `FullWireSpec` that agree on all twelve projected fields at the start agree on all of them after a shared step (`full_state_single_step_bisimulation`) and after a shared trace τ (`full_state_trace_bisimulation`). The μ /PC-only version is `three_layer_bisimulation`; its `WireSpec` requires `ws_pc (ws_step s i) = ws_pc s + 1`, which has no Coq instance because jumps violate it. Only `coq_full_wire_spec` instantiates the contract for the Coq layer. This is a contract theorem over explicit observable projections. It does not prove that the repository’s Python and Verilog artifacts satisfy `FullWireSpec`.*

23.1 RTL Gap Registry and Inventory

Theorem 23.2 (RTL gap registry and inventory (`kami_hw/RTLGapRegistry.v`), (S)). *`rtl_gap_registry_empty`: the registry of classified cases for the intermediate Gallina bridge `kami_step` lists no gaps. It is bookkeeping about that bridge, not about the synthesizable `ThieleCPUTCore.thieleCore` rule executions or the emitted Verilog. `rtl_inventory_arithmetic`: $37 + 10 + 0 = 47$, the printed classification of the 47 synthesized opcodes into 37 (31 without opcode-specific restrictions and 6 requiring valid arguments), 10 requiring structural invariants, and 0 registered gaps. Both are records, not coverage proofs: the first says what a list holds, and the second is arithmetic that mentions no opcode. The coverage evidence is the per-opcode theorems in the remark below and the step-level correspondence `driven_step_wf` under `WFDriivenPrecondition`.*

Remark 23.2. The inventory is $37 + 10 + 0 = 47$. The 37 are 31 unconditional cases (`SupportedOpcode`, `precondition True`), including `CHSH_LASSERT` (`coq/kami_hw/`

EmbedStep.v: `embed_step_compute`), and 6 cases under operand or snapshot preconditions in `WFDrivenPrecondition`: `CALL` (`WellFormedSnapshot` and `snap_pc < MEM_SIZE`), `RET` (`WellFormedSnapshot`), `CHSH_TRIAL` (`chsh_bits_ok`), `LASSERT` (`flen = lassert_hw_flen`), `TENSOR_SET` (`tensor_indices_ok`), and `TENSOR_GET` (`tensor_indices_ok` and an existing module) (`coq/kami_hw/EmbedStep_WF.v`, `coq/kami_hw/GraphReconstructionBridge.v`). The remaining 10 (`PNEW`, `PSPLIT`, `PMERGE`, `MORPH`, `MORPH_ID`, `MORPH_DELETE`, `MORPH_ASSERT`, `MORPH_GET`, `COMPOSE`, `MORPH_TENSOR`) carry Qed proofs under preconditions that differ by opcode. `MORPH_DELETE` and `MORPH_ASSERT` need `morph_table_wf`; `MORPH_GET`, `COMPOSE`, and `MORPH_TENSOR` need `extended_hw_invariant`, the conjunction of `pt_well_formed`, `morph_table_wf`, `coupling_zero_empty`, `coupling_wf`, and `coupling_desc_safe`; `MORPH` adds that invariant to module bounds and nonzero sizes; `MORPH_ID` needs `coupling_zero_empty`, a module bound, and a nonzero size; `PNEW` needs a nonempty normalized region, a module counter at most 64, and zero module tensors at the fresh slot; `PSPLIT` and `PMERGE` need `pt_well_formed` plus module-number, size, and fresh-slot conditions. Four of the invariant components are preserved by every `kami_step`: `morph_table_wf_kami_step_preserved`, `coupling_desc_safe_kami_step_preserved`, `coupling_zero_empty_kami_step_preserved`, and `coupling_wf_kami_step_preserved`. The last takes `coupling_desc_safe` as a side hypothesis (in addition to `coupling_wf`). No preservation theorem exists for `pt_well_formed` or for the operand-level premises, which remain per-instruction preconditions.

23.2 The CPU Against the Kernel

The refinement theorems relate the Kami model of the CPU to `vm_apply` under stated premises, and the two machines differ in ways those premises carry. The CPU’s data words and its ledger register are 32 bits wide, and the ledger wraps at 2^{32} ; the kernel’s words are 64 bits and its ledger is an unbounded natural number. Each refinement theorem assumes the counters it touches fit in 32 bits (`cpu_preconditions` asks for $\mu < 2^{31}$). The CPU traps with a named error code on checks the kernel does not make: `LOAD`, `STORE`, their heap forms, `CALL` and `RET` outside the active module’s range; a `PDISCOVER` whose declared cost is below its second operand; a ledger below the tensor total; malformed rich-format fields. The refinement theorems cover only steps on which none of these fires. Partition capacity and range are checked by both, with the same outcome. Retirement under the CPU’s own scheduler is proved for every admitted instruction (`admitted_retires`) and, from reset, for runs of admitted instructions (`fsm_retirement_refinement`). The board-top gate-level simulation, the SymbiYosys properties, and the board/loader equivalence check, which is conditional on the CPU interface and leaves the CPU and its RAM out, run outside Coq; no physical board has run the design.

23.3 Named Trust Boundaries

Two trust boundaries are named in Coq. The other downstream pipeline edges (extraction, the Kami printer, project text transforms, synthesis, place and route) are likewise trusted or tested rather than proved. They are not Admitted proofs. They are explicit, auditable limits of the formal claim.

bsc_kami_compilation_trusted (Layer 2, Verilog semantics gap). The Bluespec SystemVerilog compiler generates Verilog from the Kami hardware specification. The correctness of the BSC compilation step, that the generated Verilog implements exactly the Kami spec, is trusted, not proved in Coq. This is documented in `coq/kernel/hardware_bridge/VerilogRTLCorrespondence.v`, where `Definition bsc_kami_compilation_trusted` is the conjunction of two abstract propositions, `kami_pretty_printer_trusted`

and `bluespec_compiler_trusted`. The Section Variable `rtl_step_correct` states that an abstract RTL step agrees with `vm_apply` on projected state; this is the residual hypothesis that every theorem in the section quantifies over. An instance for generated Verilog additionally needs a model of that artifact and a semantic correspondence proof. The boundary is empirically backed by the cosim suite (`tests/test_verilog_cosim.py`, 38 tests, including `test_all_47_opcodes_in_cosim` which checks that every opcode appears) and the parametrized cross-layer fuzz suites (`tests/test_fuzz_random_programs.py` and `tests/test_cross_layer_adversarial_fuzz.py`).

A_QM (physical quantum axiom). No theorem proves that physical hardware implementing the kernel produces CHSH-violating statistics by exploiting quantum entanglement in hardware. The `column_contractive_iff_npa_psd` biconditional (Theorem 18.1) requires no physical axiom; it is pure algebra. The physical quantum axiom **A_QM** (declared as a `Context` parameter in `coq/PhysicsConditionalClosure.v`) is the separate named hypothesis that the chosen quantum CHSH experimental predicate admits a PSD completion with free within-party cross moments. It does not appear in the kernel-tier proof of the biconditional; it is the bridge used in conditional theorems such as `master_tsirelson_conditional`.

Chapter 24

Categorical Structure

24.1 The Morphism Layer

The active implementation adds a concrete category directly to the machine state. This is not an abstract analogy, it is an executable layer of 7 opcodes (0x27–0x2D) with formally verified laws.

24.1.1 Objects and Morphisms

The partition graph G carries two maps:

- `pg_modules` : $\text{ModuleID} \rightarrow \text{ModuleState}$, objects (partition modules)
- `pg_morphisms` : $\text{MorphismID} \rightarrow \text{MorphismState}$, arrows

A `MorphismState` bundles: source module, target module, coupling data (list of $(\text{nat} \times \text{nat})$ pairs plus a label), and an identity flag.

24.1.2 The Seven Categorical Opcodes

- `MORPH(dst, src, tgt, idx,  $\Delta\mu$ )`: create morphism $\text{src} \rightarrow \text{tgt}$, write morphism ID to `regs[dst]`. Costs $\Delta\mu$.
- `COMPOSE(dst, f, g,  $\Delta\mu$ )`: compose morphisms $f; g$ when $f.\text{target} = g.\text{source}$, write new ID to `regs[dst]`. Costs $\Delta\mu$. Errors if endpoints mismatch.
- `MORPH_ID(dst, m,  $\Delta\mu$ )`: create identity morphism id_m for module m (diagonal relation). Costs $\Delta\mu$.
- `MORPH_DELETE(f,  $\Delta\mu$ )`: remove morphism f . Cascades on module deletion via `graph_cascade_delete_morphisms`.
- `MORPH_ASSERT(f, prop, cert,  $\Delta\mu$ )`: cert-setter. Costs $S(\Delta\mu) \geq 1$ unconditionally, even for failed attempts (morphism does not exist). This is the No Free Insight law applied to relational claims.
- `MORPH_TENSOR(dst, f, g,  $\Delta\mu$ )`: tensor product $f \otimes g$. Requires that the union modules $\text{src}(f) \cup \text{src}(g)$ and $\text{tgt}(f) \cup \text{tgt}(g)$ already exist in the graph.
- `MORPH_GET(dst, f, sel,  $\Delta\mu$ )`: read field from morphism f : selector 0 = source, 1 = target, 2 = coupling length, 3 = `is_identity`.

24.1.3 Category Laws

Theorem 24.1 (Relational Composition is Associative, (S)). *In `coq/kernel/category/CategoryLaws.v`, for any three coupling lists r_1, r_2, r_3 (`relational_compose_assoc`):*

$$(r_1; r_2); r_3 \equiv r_1; (r_2; r_3)$$

where $\equiv$ is extensional equivalence of the composite coupling pairs, not equality of lists. Zero Admitted.

Theorem 24.2 (Coupling Composition Satisfies Laws, (S)). *In `coq/kernel/category/CategoryBridge.v`, the coupling-pair composite of looked-up morphisms is associative up to $\equiv$ (`morph_graph_compose_assoc`, `morph_compose_assoc_coupling`). Diagonal couplings are units up to $\equiv$ under domain and codomain premises: $\text{diag}(R); f \equiv f$ when every pair of f starts in R (`morph_id_left_coupling`), and $f; \text{diag}(R) \equiv f$ when every pair ends in R (`morph_id_right_coupling`). These are statements about `relational_compose` on coupling-pair lists; the laws for arrows stored by `graph_compose_morphisms` are Theorem 24.3. Additionally:*

$$\text{categorical_extension_nofi_consistent}$$

MORPH and COMPOSE can carry $\Delta\mu = 0$ and are not in the mandatory-cost class. The NoFI policy enters through MORPH_ASSERT: asserting that a built morphism chain exists and activating the cert channel always costs $S(\Delta\mu) \geq 1$. Building the chain is free; certifying it never is.

Theorem 24.3 (Category laws on stored arrows (`coq/kernel/category/CategoryBridge.v`), (S)). *The composite that COMPOSE stores carries the identity flag exactly when both factors carry it, and the composite of two identity arrows is stored as an identity arrow with empty coupling data and cost 0 (`graph_compose_identities_is_identity`). Call two stored arrows equivalent when they have the same source, target and identity flag and, if not identities, equivalent couplings. Then on a well-formed graph, composing $f; k$ then h and composing f with $k; h$ store equivalent arrows (`graph_compose_assoc_stored`), and an identity arrow i with equal source and target is a left and a right unit up to equivalence (`graph_compose_left_identity_stored`, `graph_compose_right_identity_stored`). From a graph with no arrows, every reachable graph stores each identity arrow with equal endpoints and no pairs (`vm_reachable_identity_arrows_canonical`).*

Theorem 24.4 (Tensor Interchange (`coq/kernel/category/CategoryMonoidal.v`), (S)). *Take the tensor of coupling lists to be list append $++$. For coupling lists $p_f, p_g, p_{f'}, p_{g'}$ whose cross composites are empty ($p_f; p_{g'} \equiv []$ and $p_g; p_{f'} \equiv []$), the interchange law holds (`tensor_bifunctor`):*

$$(p_f ++ p_g); (p_{f'} ++ p_{g'}) \equiv (p_f; p_{f'}) ++ (p_g; p_{g'})$$

up to $\equiv$. It is a statement about coupling-list append, not about `graph_tensor_morphisms` as a graph operation. `Kernel.CategoryMonoidal.monoidal_coherence` (associativity and units of `coupling_tensor`) is a separate theorem.

24.1.4 The Categorical Separation Theorem

Theorem 24.5 (Categorical Separation (`coq/kernel/foundation/PartitionSeparation.v`, §10), (S)). *There exist states s_1, s_2 such that:*

$$\begin{aligned} s_1.\text{vm_regs} &= s_2.\text{vm_regs} \wedge s_1.\text{vm_mem} = s_2.\text{vm_mem} \wedge s_1.\text{vm_mu} = s_2.\text{vm_mu} \wedge \\ s_1.\text{vm_pc} &= s_2.\text{vm_pc} \wedge s_1.\text{vm_err} = s_2.\text{vm_err} \wedge s_1.\text{vm_certified} = s_2.\text{vm_certified} \end{aligned}$$

but $s_1.\text{vm_graph.pg_morphisms} \neq s_2.\text{vm_graph.pg_morphisms}$.

Proof. Two explicit states that differ only in `pg_morphisms`: s_1 holds one identity morphism (source and target 0), s_2 holds none. Every other field is identical. Distinction by `discriminate`, on the list constructor mismatch (cons against nil). $\square$ $\square$

Remark 24.1 (Significance). A classical observer, one that sees only registers, memory, μ , PC, error flag, and certification status, cannot distinguish s_1 from s_2 (`classical_observer_cannot_separate`). `shadow_strictly_lossy` (`coq/kernel/witness/ShadowProjection.v`) gives two states with the same shadow, different morphism lists, and a graph-preserving probe that retains the distinction. No Coq statement says what `MORPH_DELETE 0` does on these two states. The categorical layer records *how* knowledge was derived, not just *what* the final values are.

This is the machine’s claim to being a *knowledge auditor*: a certifying step must have paid at least 1 (`certification_requires_positive_mu`). The NoFI theorem supplies a cost floor under its trace hypotheses. It does not authenticate an externally supplied receipt or prove arbitrary property text true.

Chapter 25

Falsifiable Predictions

The falsifiable predictions are the structural theorems that compile or fail. The claims below identify specific mathematical conditions that would constitute counterexamples to the machine's core theorems.

25.1 Falsify No Free Insight

Find a kernel step `vm_apply s i` that takes `vm_certified` from false to true, or `csr_cert_addr` from zero to non-zero, without raising μ by at least 1. The Coq proof of `certification_requires_positive_mu` would fail. The trace form is the trace-level No Free Insight theorem (`universal_nfi_any_substrate`).

25.2 Falsify the Classical CHSH Bound

Exhibit witness counts with $|S| > 2$ that are consistent with one deterministic local strategy. This contradicts `chsh_stat_violation_not_local`. Exhibit a factorizable correlator with $|S| > 2$ and it contradicts `fine_theorem` (`MinorConstraints.v`); either also contradicts known mathematics.

25.3 Falsify the Biconditional

Exhibit $E_{00}, E_{01}, E_{10}, E_{11}$ satisfying the three column-contractivity conditions but with a non-PSD NPA moment matrix. The theorem `column_contractive_iff_npa_psd` says no such example exists.

25.4 Falsify the μ -Hierarchy

Find $k \geq 1$ and a trace from `init_state` that is `level_k_certified` with `trace_mu_cost < k`. This contradicts `mu_hierarchy_theorem`.

25.5 Boundaries and conjectures

- **Physical interpretation of μ :** The two-state calorimeter protocol fixes populations, a Hamiltonian, a discrete master equation, and exact heat, but proves that the dynamics alone do not fix the Hamiltonian scale. No theorem identifies the μ ledger with thermodynamic entropy in a physical device. The calibration hypothesis is named (`mu_landauer_unruh_calibrated`) and not proved in Coq.

- **Quantum mechanics connection:** The mathematical equivalence between slice coherence and PSD of the zero-marginal NPA matrix is a kernel-tier theorem. No theorem identifies the VM with physical entanglement hardware or proves that such hardware produces CHSH-violating statistics.
- **Complexity implications:** No $\mu P \neq \mu NP$ class-separation theorem is claimed. The factored-search witness proves a concrete structural advantage under its stated model.
- **Physical constants:** No physical constant is derived from first principles. There is no derivation of Planck's constant h ; any relation tying an internal time-scale τ_μ to h would be circular by construction, defining τ_μ in terms of h and checking consistency rather than predicting h .
- **The two +1s:** the +1 in `triangle_angle`'s denominator (`PI · d_bc / (1 + d_ab + d_ac + d_bc)`, `coq/kernel/curvature/MuGravity.v`) versus the A2 cost-floor +1 on `CERTIFY/LASSERT` (`coq/kernel/foundation/VMStep.v`), same axiom, or unrelated regularisation? No kernel theorem identifies them (`coq/kernel/frontier/TriangleAnglePlusOne.v`). That file proves `instr_certify_cost_is_successor`, which is `instruction_cost (instr_certify cost) = S cost` and says nothing about `triangle_angle`, and `triangle_angle_plus_one_correction_decays`: the correction term decays as $-\pi/(3(3d+1))$ in the equilateral case (`tools/triangle_angle_plus_one_analysis.py` computes it symbolically). The kernel uses the stated denominator; no identification theorem is claimed.
- **Non-vacuity of the calibrated angle-defect theorem:** refuted. Calibration at every module is impossible on a well-formed triangulated partition graph whose vertex links are connected and whose module identifiers are distinct (`connected_triangulation_not_calibrated`); the hypotheses are met by a concrete state (`calibration_obstruction_hypotheses_satisfiable`). Calibration forces every face flat (`calibration_forces_flat_faces`), the +1 in `triangle_angle` bounds the triangle count against the face count, and an Euler bound on each component, a set that contains a module exactly when the module is reachable from it through shared edges (`euler_component`), contradicts the boundary count that `well_formed_triangulated` fixes. The headline of `PartitionTopologyCrossLink.v` is therefore vacuous on such graphs, and on states the VM reaches it is vacuous for every graph with a module (`reachable_calibrated_iff_no_modules`). No theorem derives the asymptotic behavior from the +1.
- **Cross-base equivalence** (`coq/kernel/foundation/CrossBaseGranularity*.v`): Base adapters for the Turing machine (`tm_base`), the VM (`vm_base`), a Cook–Reckhow unit-cost RAM (`ram_base`), and L (`l_base`) exist, and each satisfies `record_axis_is_latch_on` (`record_axis_is_latch_on_tm_holds`, `record_axis_is_latch_on_vm_holds`, `record_axis_is_latch_on_ram_holds`, `record_axis_is_latch_on_l_holds`); `record_axis_is_latch_on` holds for every base, so these instances do not distinguish the bases. The substantive adapter facts are that the L and RAM bases are faithful (`l_step_fun_correct`, `star_is_l_base_run`). The relation `weak_base_equiv` is reflexive, symmetric, and transitive, and it carries no cost. Cost-respecting simulations between different bases are not constructed, and no `weak_base_equiv` between two different bases is proved.
- **Cost-framework comparison:** The graded, writer, potential, and counting theorems are local correspondences. Full semantics-preserving embeddings of the four cost-framework families have not been constructed.

Appendix A

Proof File Index

The table maps theorem families to their Coq source files in the active build. Observation adequacy, joint event floors, and abstract calibration consistency are in `coq/kernel/frontier/ObservationPolicy.v`; descent from traces and its concrete targets are in `coq/kernel/frontier/TraceStateDescent.v`.

Result	Coq Source
<i>Cost ledger and No Free Insight</i>	
No Free Insight (Thm. 6.1)	<code>coq/nofi/NoFreeInsight_Theorem.v</code>
μ -Chaitin	<code>coq/nofi/MuChaitinTheory_TraceLocal.v</code> , <code>coq/nofi/MuChaitinTheory_Theorem.v</code>
μ -Conservation	<code>coq/kernel/foundation/MuLedgerConservation.v</code>
Receipt Integrity	<code>coq/kernel/nfi/ReceiptIntegrity.v</code>
Noether / Gauge	<code>coq/kernel/curvature/KernelNoether.v</code>
Strict Subsumption	<code>coq/kernel/foundation/Subsumption.v</code>
Universal NFI (any substrate)	<code>coq/kernel/nfi/UniversalCertificationCost.v</code> : <code>universal_nfi_any_substrate</code>
Quantitative NFI ($\geq S(d)$ trace bound)	<code>coq/kernel/mu_calculus/QuantitativeNoFI.v</code>
μ -Hierarchy	<code>coq/kernel/mu_calculus/MuHierarchyTheorem.v</code> : <code>mu_hierarchy_theorem</code>
Earned commitments (minimal machine)	<code>minimal/EarnedCore.v</code> , <code>coq/kernel/foundation/EarnedCoreLinks.v</code>
Partition bounds on reachable states	<code>coq/kernel/foundation/VMStep.v</code> : <code>vm_reachable_partition_in_bounds</code>
Geometry on reachable states	<code>coq/kernel/frontier/ReachableGeometry.v</code>
Partition Ops Free, Certification Non-Free	<code>coq/kernel/nfi/PartitionRefinementNoFI.v</code>
μ -Ledger Necessity (5-way classification)	<code>coq/NecessityOfMuLedger.v</code> , <code>coq/kernel/nfi/NecessityAbstract.v</code>
Trace Cost for a Fixed Schedule	<code>coq/NecessityOfMuLedger.v</code> : <code>shadow_mu_is_computation_intrinsic</code>
μ Cost Difference Independent of Starting State	<code>coq/NecessityOfMuLedger.v</code> : <code>shadow_mu_delta_universal</code>
μ Cost Unique Across Accounting Systems	<code>coq/NecessityOfMuLedger.v</code> : <code>shadow_mu_unique_accounting</code>
Positive-Cost Trace Accumulation	<code>coq/NecessityOfMuLedger.v</code> : <code>shadow_mu_inevitable</code>
μ -budget predicates + zero- μ inclusion	<code>coq/kernel/mu_calculus/MuComplexity.v</code>
<i>Categorical and shadow-projection layer</i>	
Categorical Separation / Category Laws	<code>coq/kernel/foundation/PartitionSeparation.v</code> , <code>coq/kernel/category/CategoryLaws.v</code> , <code>coq/kernel/category/CategoryBridge.v</code>

Shadow Strictly Lossy	coq/kernel/witness/ShadowProjection.v: shadow_strictly_lossy
Turing Strictness	coq/kernel/foundation/TuringStrictness.v
CHSH Coupling Bridge ($S \leq 2$ from local coupling, $S > 2$ rules it out)	coq/kernel/quantum/CHSHCouplingBridge.v
Witness Insight Three-Tier	coq/kernel/witness/WitnessInsightGeneral.v
Taxonomy	
<i>Quantum / CHSH algebra (no project-local axioms)</i>	
Slice-coherent $\iff$ NPA PSD (biconditional)	coq/kernel/quantum/QuantumPartitionPSD.v: column_contractive_iff_npa_psd
Forward: column-contractive $\Rightarrow$ NPA PSD	coq/kernel/nfi/MuLedgerQuantumBridge.v: zero_marginal_npa_column_contractive_implies_psd
Reverse: NPA PSD $\Rightarrow$ column-contractive	coq/kernel/quantum/QuantumPartitionPSD.v: npa_psd_implies_column_contractive
Quadratic-form discriminant lemma	coq/kernel/quantum/ConstructivePSD.v: quadratic_nonneg_discriminant
Tsirelson from column contractivity ($ S \leq 2\sqrt{2}$)	coq/kernel/nfi/MuLedgerQuantumBridge.v: state_column_contractive_implies_tsirelson (tsirelson_from_row_bounds in TsirelsonGeneral.v is the inner Cauchy–Schwarz step)
Tsirelson rational lower witness	coq/kernel/category/AlgebraicCoherence.v: tsirelson_rational_lower_witness
Tsirelson (algebraic from coherence)	coq/kernel/quantum/TsirelsonFromAlgebra.v
General Algebraic Tsirelson ($S^2 \leq 8$)	coq/kernel/category/AlgebraicCoherence.v: algebraically_coherent_tsirelson_general
Elliptope realizability (existential completion)	coq/kernel/quantum/ElliptopeCompletion.v: elliptope_realizable (definition); deterministic_strategy_elliptope, lhv_mixture_elliptope (every LHV correlator inside)
Tsirelson for the full elliptope ($S^2 \leq 8$)	coq/kernel/quantum/ElliptopeCompletion.v: elliptope_tsirelson (inner step psd_cauchy_schwarz)
PR box excluded; classical $\subset$ elliptope strict	coq/kernel/quantum/ElliptopeCompletion.v: pr_box_not_elliptope, beyond_classical_elliptope
Decidable elliptope gate (soundness)	coq/kernel/quantum/ElliptopeGate.v: elliptope_check_full_sound; boundary acceptance gate_accepts_pythagorean_boundary; universal refusal elliptope_full_gate_never_accepts_pr_box
<i>Frontier: pointer-observable criterion</i>	
Proliferation / unique-pointer schema	coq/kernel/frontier/PointerObservable.v: redundantly_proliferating, unique_pointer_among (definitions); toy_cert_unique_pointer (sanity instance)
Models of the five disciplines are unique pointers	coq/kernel/frontier/PointerObservableReductions.v: PoS_model_unique_pointer, Gas_model_unique_pointer, TEE_model_unique_pointer, CT_model_unique_pointer, PCC_model_unique_pointer; conjunction five_labeled_models_have_selected_pointer (closed under the global context). Correspondence to the deployed disciplines is argued in prose, not proved
Adversarial search: forgery-resistant systems that do <i>not</i> proliferate	coq/kernel/frontier/PointerObservableCounterexamples.v: deniable_authentication_model_not_proliferating, mac_model_not_proliferating, capability_model_not_proliferating; control public_log_model_proliferating. The Coq results fix the observer maps; (C1) and (C2) are argued in prose
<i>Conditional physics derivations (under named axioms; (C) tier)</i>	
Born Rule	coq/kernel/quantum/BornRule.v
Unitarity	coq/kernel/quantum/Unitarity.v
Purification	coq/kernel/quantum/Purification.v

No-Cloning	coq/kernel/quantum/NoCloning.v
Information Causality	coq/kernel/quantum/InformationCausality.v
Landauer derivation	coq/thermodynamic/LandauerDerived.v
Affine Off-Diagonal Ricci Zero (vertex 1)	coq/kernel/curvature/AffineEFEClosure.v
Full Tensor EFE at Vertex 1 ($G = 3T$)	coq/kernel/curvature/AffineEFEClosure.v
Outer Vertex Characterization (Ricci = $3/32$)	coq/kernel/curvature/AffineEFEClosure.v
Spacetime Emergence (consistency check)	coq/kernel/curvature/SpacetimeEmergence.v
μ -Geometry (pseudometric)	coq/kernel/mu_calculus/MuGeometry.v
4D Simplicial Complex	coq/kernel/curvature/FourDSimplicialComplex.v
Metric from μ -Costs	coq/kernel/curvature/MetricFromMuCosts.v
Riemann / Christoffel / Einstein tensors	coq/kernel/curvature/RiemannTensor4D.v
Stress-Energy Tensor / Conservation / Vacuum EFE	coq/kernel/curvature/EinsteinEquations4D.v
Einstein Equations (general, full diagonal-and-offdiag chain)	coq/kernel/curvature/EinsteinEquationsFull.v
A_QM bridge axiom + master Tsirelson conditional	coq/PhysicsConditionalClosure.v
<i>Self-reference, falsifiability, observable agreement</i>	
Self-Reference Tower	coq/self_reference/SelfReference.v
Thiele Manifold	coq/thiele_manifold/ThieleManifold.v
Physical Constants (definitions only)	coq/thiele_manifold/PhysicalConstants.v
Falsifiable Prediction record	coq/kernel/aggregators/FalsifiablePrediction.v
Three-Layer (Coq $\leftrightarrow$ Python $\leftrightarrow$ RTL)	coq/kernel/hardware_bridge/ThreeLayerIsomorphism.v
Observable Agreement	
<i>Hardware / RTL / extraction</i>	
Gap registry of the intermediate Gallina bridge empty	coq/kami_hw/RTLGapRegistry.v: rtl_gap_registry_empty
RTL inventory count (37 + 10 + 0 = 47, arithmetic only)	coq/kami_hw/RTLGapRegistry.v: rtl_inventory_arithmetic
RTL Trust Boundary (Section Variable)	coq/kernel/hardware_bridge/VerilogRTLCorrespondence.v: rtl_step_correct
BSC Compilation Trust Boundary	coq/kernel/hardware_bridge/VerilogRTLCorrespondence.v: bsc_kami_compilation_trusted
Gallina <code>kami_step</code> agrees with <code>vm_apply</code> for <code>SupportedOpcode</code> (<code>coq_kami_model_satisfies_rtl_step_correct</code>), and under <code>WFDDrivenPrecondition</code> for the rest; not a semantics of generated Verilog	coq/kami_hw/VerilogSemantics.v
<code>EmbedStep</code> / <code>EmbedStep_WF</code> (per-opcode bisimulation)	coq/kami_hw/EmbedStep.v, coq/kami_hw/EmbedStep_WF.v
<code>GraphReconstructionBridge</code> (<code>TENSOR_*</code> , <code>LASSERT</code>)	coq/kami_hw/GraphReconstructionBridge.v
ThieleCPU Kami specification	coq/kami_hw/ThieleCPUCore.v
OCaml extraction observable surface	coq/kernel/hardware_bridge/OCamlExtractionBridge.v: ocaml_observable_nofi_and_monotone

Appendix B

Epistemological Classification

The following table classifies every major result by its epistemological status.

Remark B.1 (How to read the tags). This table is the document’s principal credibility instrument, so four classifications that a reader is most likely to trip over are stated explicitly here.

- **Tsirelson** carries two entries that are not the same result. The *algebraic* bound is **(S)**: pure polynomial arithmetic, zero physics axioms. What is conditional is *physical realizability*, the **A_QM** bridge, which is its own line item and should never be folded into the algebra.
- **Born rule** likewise carries two entries for two results. *Born Rule Uniqueness* is **(C)**, conditional on the linearity assumption. The **BornRule.v** relation is **(R)**, because its measurement-cost hypothesis goes unused in the proof.
- **Affine EFE** is **(R)**: reference-only geometry. The containing chapter is discrete geometry on a non-negative per-module tensor with no Lorentzian signature asserted, so there is no causal structure to make a relativistic claim about (Remark 19.4).
- **μ -Chaitin** is **(C)**. The interface **MuChaitinTheory_Interface** is a module-type contract with no VM instance, because the VM schedule is not globally cert-priced. The trace-local interface **MU_CHAITIN_TRACE_SYSTEM** has one VM instance, **KernelTraceInstance** (**MuChaitinTheory_TraceLocal.v**). The general bound stays conditional on the interface obligations, among them the description-length budget; the instance bound **kernel_trace_instance_bound** is a closed statement about one concrete run. The template for discharging an interface is **Instance_Kernel.v**, which does exactly this for NFI.

Result	Status	Key Assumption
<i>Unconditional structural theorems about the machine</i>		
No Free Insight	(S)	Module type contract
μ -Chaitin	(C)	Module-type contract. MuChaitinTheory_Interface has no VM instance; the trace-local MU_CHAITIN_TRACE_SYSTEM has the concrete instance KernelTraceInstance . The general bound rests on the interface obligations, including the description-length budget (cf. Instance_Kernel.v for the NFI template)
Strict Subsumption	(S)	Partition structure
Oracle Cost (instantiation of universal NFI)	(S)	Definitional (soundness := cost ≥ 1)
μ -Conservation	(S)	Machine semantics only
μ -Ledger Necessity	(S)	Machine semantics + explicit witnesses

μ -Ledger Irredundancy (neither μ nor cert droppable)	(S)	Quantifies over all carriers/projections. <i>Not</i> minimality: no order on projections is defined, so nothing is proved least (theorem <code>P_full_complete_neither_mu_nor_cert_droppable</code>)
Receipt Integrity	(S)	Hash-chain model
Gauge / Noether	(S)	Definitional (record structure)
Three-Layer Observable Agreement (<i>not</i> an isomorphism: no pair of mutually inverse maps is constructed; the content is transitivity of equality on a shared projection)	(C)	Backend decoders satisfy the shared observable projection contract
Discrete Angle-Defect Identity (planar triangulations)	(S)	Well-formed triangulated partition graph; equilateral model; not classical Gauss-Bonnet
Curvature from μ -Gradients (uniform module tensors $\Rightarrow G_{\mu\nu} = 0$)	(R)	Reference only; $0 = 0$ in vacuum
Vacuum Implies Zero Stress-Energy	(R)	Reference only; $0 = 0$ in vacuum
Einstein Equations (Vacuum)	(R)	Reference only; not a claim about physics; $G = 1/(8\pi)$ is a definition
Einstein Equations (General full tensor)	(C)	Named off-diagonal Ricci premise in <code>EinsteinEquationsFull.v</code>
Affine Off-Diagonal Ricci Zero (vertex 1)	(R)	Reference only; specific boundary 4-simplex configuration
Full Tensor EFE at Vertex 1 ($G = 3T$)	(R)	Reference only; not a physical claim
Outer Vertex Characterization	(R)	Reference only
Discrete Differential Geometry	(R)	Definitions only; reference exploration
Partition Ops Free / Certification Non-Free	(S)	Machine semantics only
CHSH Trial Is Tier 0	(S)	Machine semantics only
Three-Tier Observation Taxonomy	(S)	Machine semantics only
Locally Consistent $\Rightarrow$ Separable Coupling	(S)	Coupling definition
Local Coupling Bound ($ S \leq 2$)	(S)	Coupling definition
$S > 2$ Rules Out Local Couplings	(S)	Coupling definition
RTL Bisimulation (<code>driven_step_wf</code>)	(C)	Per-opcode agreement of the intermediate Gallina model <code>kami_step</code> with <code>vm_apply</code> under the explicit premise <code>WFDDrivenPrecondition</code> . (<code>rtl_inventory_arithmetic</code> , $37+10+0 = 47$, is Peano arithmetic and is <i>not</i> coverage evidence)
Slice-Coherent $\iff$ NPA PSD	(S)	Polynomial arithmetic; standard-library assumptions
PR Box Not Slice-Coherent	(S)	Direct arithmetic on condition 1
Elliptope Realizability (full correlator set)	(S)	Existential completion; stdlib reals only
Decidable Elliptope Gate Sound	(S)	Fraction-free $\mathbb{Z}$ arithmetic, two branches
Five Disciplines' Models Are Unique Pointers	(S)	Closed under the global context; model structure only. Evidence for (PO-PUBLIC) only through the prose correspondence; see Section 18.3.1

Three Candidate Counterexamples to PO-STRONG	(R)	Coq checks the three observer maps (deniable authentication, symmetric MAC, object capabilities); their faithfulness to the real designs, and so the refutation, is a prose modeling judgment
Tsirelson Is Thiele Boundary (forward + contrapositive)	(S)	From column contractivity; converse not claimed
Zero- μ traces satisfy the preservation budget	(S)	Error-free traces; no returned-answer or uniform-solver requirement
Forced price is merging; merge or revoke	(S)	Finite enumeration; decidable instruction equality for the iff; converse to permanence fails on three states
Entropy drop of a permanent flip	(S)	Finite enumeration, permanent certificate; Shannon entropy over Coq reals; standard real-number axioms only
Finite VM instance	(S)	Eight-state machine; VM fragment <code>CERTIFY 0, JUMP a 2, CHECKPOINT</code> through the window (<code>pc mod 4, certified</code>)
Full VM does not price every merge	(S)	Every certification flip is a priced merge; <code>JUMP 0 0</code> is a free merge; other merges are not classified
No exact shadow price	(S)	Any window with a collision; VM collisions for <code>bare_observable</code> and <code>forget</code>
<i>Conditional derivations (valid given stated axioms)</i>		
Born Rule Uniqueness	(C)	Linearity assumption (<code>coq/kernel/quantum/BornRule.v</code> , <code>BornRuleLinearity.v</code>)
Unitarity from $\mu = 0$	(C)	Info conservation and purity-nonincreasing premises; converse not proved
No-Cloning	(C)	Purity = information; conservation inequality supplied as a premise
Purification	(C)	Bloch sphere model
Landauer	(C)	Second law <i>assumed</i> as record field
A2 from a merging price	(C)	Finite enumeration, permanent certificate, non-injective steps cost ≥ 1 (Landauer premise); core merge theorem is (S)
Cost of a squeeze	(C)	Counting premise: cost c allows at most a 2^c -fold squeeze; bound attained by a four-state stamp
Heat floor of a permanent flip	(C)	Landauer's principle as the named premise <code>landauer_heat</code>
Self-Reference	(C)	Dimension = complexity; the dimension is a record field chosen by definition
<i>Consistency relations (definitions verified, not predictions)</i>		
Information Causality	(R)	Circular: equivalence assumed as hypothesis
μ -Geometry	(R)	Pseudometric (absolute value on $\mathbb{Z}$)
Spacetime Locality	(R)	Data isolation in partition graph
Born Rule (<code>BornRule.v</code>)	(R)	Measurement-cost hypothesis unused

Legend: (S) = unconditional theorem about the machine; (C) = valid derivation conditional on stated axiom; (R) = internal consistency check.

Exact scope

The record-axis theorems have an exact scope: a permanent record driven by the base computation is a latch on an event, a finite permanent write is non-injective, and the VM's named projections do not recover its record. Certification is an instance; no theorem derives a unique event choice.

Finite monotone multi-valued records admit threshold-latch decompositions, but the corresponding single-latch, revocable, and probabilistic uniqueness statements are false. The guest VM has a direct Rice reduction, numeric decoding, verified fuel-bounded dispatch, semantic s-m-n specialization, and its own recursion theorem (`vm_guest_recursion_theorem_closed`):

every program transformer that a guest program computes on program codes has a fixed point that matches it in all four final registers and in μ on every input, halting or not. The evaluator in that proof is extracted to the lambda calculus L, compiled to a Minsky machine, and executed by guest code. For the full VM, a fixed point for every map on programs up to equal thousand-step runs is false. The 51-opcode VM prices commitments without checking that they were earned; the minimal two-counter machine earns them. The geometry theorems are about partition graphs in general; on states the VM reaches, module regions are disjoint and calibration holds only with no modules. The two-state calorimeter computes exact heat and proves that identical population dynamics do not determine the Hamiltonian scale. Converting μ to energy requires thermal and device premises; μ has no intrinsic energy unit.

The RFC 9162 file checks iterative inclusion and consistency control flow and examples, not cryptographic collision resistance or signed-tree-head authenticity. The PCC, addressed-RAM, reversible-update, and TPM-interface files prove only their named fragment or countermodel propositions. The local cost-semantics correspondences are not full cost-framework embeddings.

The twelve-event observer-map bundle and swap theorem are closed. Its MAC-labelled model is non-proliferating, but applying that statement to a real system depends on the observer model. The two-observer toggle game refutes the claim that consensus, exact observation, and coordinator-free evolution force permanence; the durability theorem supplies persistence as a premise, and the VM certification game satisfies it (`vm_certification_is_permanent_consensus`). The external consequence survey contains five narrow statements already known from their governing specifications and five stronger statements without enough protocol or hardware semantics. No surveyed consequence is simultaneously record-axis dependent, novel, and ready for use. The pointer criterion is a conjecture.